



Trinity College Dublin

Coláiste na Tríonóide, Baile Átha Cliath

The University of Dublin

Communication-Avoiding Krylov Subspace Methods for Option Pricing

In partial fulfillment of MSc in High-performance Computing
in the School of Mathematics

Name: Kevin Knights
Student ID: 25378095
Supervision: Kirk M. Soodhalter

Date of Submission: Date you submitted your thesis
Project Code: <https://github.com/kevinknights29/caksm>



Trinity College Dublin
Coláiste na Tríonóide, Baile Átha Cliath
The University of Dublin

Declaration concerning plagiarism

I hereby declare that this thesis is my own work where appropriate citations have been given and that it has not been submitted as an exercise for a degree at this or any other university.

I have read and I understand the plagiarism provisions in the General Regulations of the University Calendar for the current year, found at <http://www.tcd.ie/calendar>.

Name: Kevin Knights

Student ID: 25378095

Signature: 
.....

Date: Date your submitted your thesis

ACKNOWLEDGEMENTS

Write acknowledgements to your supervisor, classmates, friends, family, partner... anyone who supported you during the MSc.

ABSTRACT

A trading desk repricing a multi-asset option book revalues the same partial differential equation thousands of times a day, and at production resolution a single price costs on the order of hundreds of seconds. That cost is dominated by communication, not by arithmetic: a Krylov exponential integrator spends over 70% of its time in sparse matrix–vector products that move far more bytes than they compute, and most of the rest in an orthogonalization whose every step is a global synchronization. Communication-avoiding (CA) reformulations attack exactly these two costs by spending redundant arithmetic to buy fewer memory-hierarchy crossings and fewer synchronizations—an exchange whose benefit is not guaranteed, because it depends on the operator, the grid, and the machine.

This thesis answers *when* the exchange pays, in coordinates that transfer. A benchmark pricer implementing five methods on two 3-asset European options establishes an uncompromised baseline and reproduces the reference results of Niesen and Wright [19]. A regime study then builds a dimensionless map of the CA design space: a vertical axis R_v (working set over aggregate last-level cache) governing matrix-powers tiling, and a horizontal axis R_h (reduction cost over compute between reductions) governing s -step orthogonalization, with predicted crossovers at 1. The coordinates are predictors in the strict sense: both are computed from the problem size, the sparsity pattern, the spectrum, and offline-calibrated machine constants before any timed run, so a measurement can confirm or refute a placement but can never be fitted into one.

The numerical results certify the instrument. The s -step block width certified by the CholeskyQR2 stability theorem $\kappa(\mathbf{B}) \leq u^{-1/2}$ [23] is predicted from the spectrum alone via a row-scaled Vandermonde argument and verified integer-exact on 20 of 21 runs; an empirically fitted log-robustness relation bounds the damage non-normality does to that prediction; and a closed-form tensor identity gives the growth of eigenvector conditioning with asset dimension. The measured results place the real operator. Cutting the reduction count 11–16 $\times$ leaves Gram–Schmidt wall-clock unchanged—exactly as the map predicts at $R_h = 0.080$, because the kernel is bandwidth-bound, not synchronization-bound. Cache-blocked matrix powers trades halo arithmetic for DRAM traffic at a loss, a kernel ceiling that leaves the vertical boundary untested rather than refuted. And the production operator’s trajectory under grid refinement lands squarely in vertical-only territory: matrix-powers tiling is the mechanism the application needs. Finally, the map’s portability is tested on a GPU memory system that differs in kind: recomputed—not refitted—coordinates place the production operator in an upper-right corner the single-socket CPU could not reach, and a roofline gate correctly predicts that the s -step arm’s own orthogonalization kernel pays on a datacenter GPU while turning compute-bound on a consumer one.

Keywords: Krylov subspace methods, communication-avoiding algorithms, matrix exponential, option pricing, roofline model, s -step methods, performance modeling

CONTENTS

1	Introduction	6
2	The pricing problem and its discretization	6
3	The Krylov exponential integrator	8
	3.1 Cost structure and the sequential dependency	8
	3.2 Communication-avoiding reformulations	8
4	Experimental platform and methodology	10
5	Benchmark: reproducing Niesen–Wright	11
6	Profile and roofline	12
7	The OpenMP scaling study	13
	7.1 Instrument design	13
	7.2 The quarantined $m(n)$ growth	15
	7.3 The locality control	15
8	Regime analysis	16
	8.1 The coordinates	17
	8.2 The instrument and its gates	18
	8.3 Basis conditioning and the confound control	18
	8.4 The reduction ledger and small- m safety	19
	8.5 Non-normality: the log-robustness relation	20
	8.6 Conditioning growth with asset dimension	22
	8.7 The computability gap	23
	8.8 The horizontal mechanism	24
	8.9 The vertical mechanism	25
	8.10 Block width	26
	8.11 Placing the real operator on the map	27
9	Recomputing the map for a GPU	29
	9.1 The portability claim under test	29
	9.2 What transfers, and what breaks	30
	9.3 The decoupled invariant	30
	9.4 Reaching the corner, and the gate that discriminates	31
	9.5 Roofline-bound kernels and the s -step crossover	32
10	Conclusion	33
	References	35

1 INTRODUCTION

A trading desk repricing a multi-asset book revalues the same PDE thousands of times a day, so the wall-clock cost of one solve sets what can be quoted, hedged, and risk-managed intraday. For a 3-asset basket at a resolution fine enough to matter, the published cost is roughly 700 s per price [19]. That cost is dominated by communication, not by arithmetic. A Krylov exponential integrator spends over 70% of its time in sparse matrix–vector products (SpMV) that move far more bytes than they compute, and the rest in an orthogonalization whose every step is a global synchronization. Both are *communication* costs: data crossing the memory hierarchy, and cores waiting on each other [3, 15]. Communication-avoiding (CA) reformulations attack exactly these—matrix powers computes s Krylov vectors per pass over the operator instead of one [8, 18], and s -step orthogonalization replaces $O(m^2)$ reductions with $O(m/s)$ [15, 6]. Both spend redundant arithmetic, which is cheap, to buy fewer hierarchy crossings and fewer synchronizations, which are not.

Whether that exchange pays is not a property of the algorithm alone: it depends on the operator, the grid, and the machine. This thesis answers *when* it pays, and does so in coordinates that transfer.

Two artifacts support that answer. A **benchmark pricer** in C++23 implements five methods (Crank–Nicolson, two ADI variants, a one-shot matrix exponential, and a Krylov exponential integrator) on two 3-asset European options, and establishes the uncompromised baseline the CA work must beat (Section 5). A **regime study** (Section 8) builds a dimensionless map of the CA design space: because both of its axes are ratios rather than absolute times, the map is independent of the machine and of the problem, so a reader can place their own operator and hardware on it and read off which mechanism, if either, can pay for them.

One design rule keeps the map non-circular, and everything else depends on it: *coordinates are predictors*, computed before a run from the problem size, the sparsity pattern, the spectrum, and the machine constants; *outcomes* (wall-clock, orthogonality loss, achieved bytes) test the boundaries and never enter a coordinate. A coordinate that ingested a measurement could always be tuned until the crossover landed at 1, and the map would prove nothing.

The thesis proceeds as follows. Section 2 states the pricing problem and its discretization. Section 3 presents the Krylov exponential integrator, its cost structure, and the CA reformulations that target it. Section 4 describes the experimental platform and measurement methodology. Section 5 reproduces the Niesen–Wright benchmark as a control. Section 6 profiles the solver and places its kernels on a cache-aware roofline. Section 7 presents the OpenMP scaling study that motivates the two CA mechanisms. Section 8 develops the regime map and its numerical and measured results. Section 9 recomputes the coordinates for a GPU memory system—a structural change rather than a constant swap—and tests the map’s portability claim by measurement on a controlled pair of GPUs. Section 10 concludes.

2 THE PRICING PROBLEM AND ITS DISCRETIZATION

The model problem is the 3-asset Black–Scholes PDE [5] in log-price coordinates, solved backward in pseudo-time τ from 0 (payoff) to T (price):

$$\frac{\partial u}{\partial \tau} = \frac{1}{2} \sum_{d,d'} \rho_{dd'} \sigma_d \sigma_{d'} \frac{\partial^2 u}{\partial x_d \partial x_{d'}} + \sum_d \left(r - \frac{1}{2} \sigma_d^2 \right) \frac{\partial u}{\partial x_d} - r u + b(\tau). \quad (2.0.1)$$

The three groups of terms recur throughout the thesis under their standard PDE names. The second-derivative terms are *diffusion* (volatility spreading value across the surface; the mixed $d \neq d'$ terms cou-

ple correlated assets). The first-derivative terms are *convection*, also called *advection* (the risk-neutral drift transporting value across the grid). The zeroth-order term $-ru$ is *reaction* (here, discounting at the risk-free rate).

Two payoffs are priced. The basket call,

$$u_0(x) = \max\left(\sum_d w_d e^{x_d} - K, 0\right),$$

and the rainbow min-call,

$$u_0(x) = \max\left(\min_d e^{x_d} - K, 0\right).$$

The boundary forcing term $b(\tau)$ for the basket case is encoded as a polynomial $B \cdot s(\tau)$ with $s(\tau) = [\tau^2/2, \tau, 1]^\top$, derived from a deep in-the-money approximation on ghost nodes outside the grid. The rainbow case uses modified finite-difference stencils (a zero-gamma boundary condition) instead.

Discretization is by second-order central differences on a uniform Cartesian grid with x_1 -fastest lexicographic ordering (grid points are numbered consecutively along x_1 first, then x_2 , then x_3), so every one-dimensional operator extends to the full grid as a three-fold Kronecker product with identity in the undifferentiated directions. The assembled $\mathbf{A}$ carries at most 19 nonzeros per row (a 7-point stencil plus 3×4 mixed-derivative diagonals). Its bandwidth is $O(N_1 N_2)$, set by the x_3 and mixed terms, which rules out banded direct solvers at scale. $\mathbf{A}$ splits as $\mathbf{A}_{\text{sym}} + \mathbf{A}_{\text{skew}}$ —diffusion, mixed, and reaction terms symmetric, convection skew-symmetric—so it is not symmetric in general, but every eigenvalue satisfies $\text{Re}(\lambda) \leq 0$, which is what stability of $e^{h\mathbf{A}}$ requires [16]. Table 1 lists the default model parameters used throughout.

Table 1: Default model parameters.

Parameter	Value	Description
Spot prices S_0	(100, 100, 100)	Initial asset prices
Strike K	100	Option strike
Risk-free rate r	0.04	Continuously compounded
Maturity T	1.0 year	Fixed
Volatilities σ	(0.30, 0.35, 0.40)	Per-asset annual volatility
Correlations ρ	(0.50, 0.50, 0.50)	Off-diagonal pairs $(\rho_{01}, \rho_{02}, \rho_{12})$
Basket weights w	(1/3, 1/3, 1/3)	Equal-weight basket
Grid half-width α	2.85	Domain $\pm\alpha\sigma\sqrt{T}$ per dimension

Stiffness grows quadratically with refinement, and the discretization says why. The central second difference of the diffusion term on n interior points per axis has eigenvalues

$$\lambda_k = -\frac{2\sigma^2}{\Delta x^2} \left(1 - \cos \frac{k\pi}{n+1}\right), \quad k = 1, \dots, n,$$

so the extreme eigenvalue scales as $O(\Delta x^{-2}) = O(n^2)$ [16]. Measured on the assembled operator, the smallest eigenvalue moves from -24 at grid size $n = 11$ to -2924 at $n = 101$ (where $N = n^3$ is the total number of unknowns), consistent with that scaling. Over-solving in time on a coarse grid is therefore pointless, and the commercial motivation sits at the other end: Niesen and Wright report ~ 700 s for Hundsdorfer–Verwer to reach one basis point at 61^3 [19].

3 THE KRYLOV EXPONENTIAL INTEGRATOR

After spatial discretization, the semi-discrete system takes the affine form

$$\frac{d\mathbf{u}}{d\tau} = \mathbf{A}\mathbf{u} + \mathbf{b}, \quad \mathbf{u}(\tau_0) = \mathbf{u}_0. \quad (3.0.1)$$

The constant forcing vector $\mathbf{b}$ makes the system *inhomogeneous*: the right-hand side is no longer a pure linear map of the state, so $e^{h\mathbf{A}}\mathbf{u}_k$ alone does not advance the solution. Following Niesen and Wright [19], the inhomogeneity is absorbed into an *extended system*: define

$$\tilde{\mathbf{A}} = \begin{bmatrix} \mathbf{A} & \mathbf{b} \\ \mathbf{0}^T & 0 \end{bmatrix} \in \mathbb{R}^{(N+1) \times (N+1)}, \quad \tilde{\mathbf{v}} = \begin{bmatrix} \mathbf{u}_k \\ 1 \end{bmatrix} \in \mathbb{R}^{N+1}, \quad (3.0.2)$$

so that the exact solution at time $\tau_0 + h$ is encoded in the first N components of $e^{h\tilde{\mathbf{A}}}\tilde{\mathbf{v}}$.

The Krylov approximation proceeds via the Arnoldi iteration on $\tilde{\mathbf{A}}$ with starting vector $\tilde{\mathbf{v}}$, producing an orthonormal basis $\mathbf{V}_m$ and an upper Hessenberg matrix $\bar{\mathbf{H}}_m \in \mathbb{R}^{(m+1) \times m}$ [20, 14]. Dropping the last row gives $\mathbf{H}_m \in \mathbb{R}^{m \times m}$, and the approximation reads

$$e^{h\tilde{\mathbf{A}}}\tilde{\mathbf{v}} \approx \beta \mathbf{V}_m e^{h\mathbf{H}_m} \mathbf{e}_1, \quad \beta = \|\tilde{\mathbf{v}}\|. \quad (3.0.3)$$

Algorithm 1 gives the full adaptive procedure, and Algorithm 2 the Arnoldi iteration it calls.

3.1 COST STRUCTURE AND THE SEQUENTIAL DEPENDENCY

The dominant cost of the integrator is the Arnoldi iteration. Each step j requires one sparse matrix–vector product $\mathbf{w} = \tilde{\mathbf{A}}\mathbf{v}_j$, costing $O(N)$, and orthogonalization against all previous vectors, costing $O(jN)$. Crucially, step j cannot begin until $\mathbf{v}_j$ is available, which requires step $j-1$ to have completed. This creates a strictly sequential data dependency across the Krylov dimension m . The individual SpMV and inner products within each step can be parallelized, but this inner parallelism scales poorly: the SpMV is bandwidth-bound, and every inner product in the modified Gram–Schmidt (MGS) loop is a global reduction. These are the two *communication* walls (Table 2), and they are different in kind.

Table 2: The two kernels, their bounds, and the CA mechanism each motivates.

Kernel	Bound	Communication	CA mechanism
SpMV	bandwidth (memory hierarchy)	vertical	matrix powers / cache blocking
Gram–Schmidt	synchronization (reductions)	horizontal	s -step / block orthogonalization

3.2 COMMUNICATION-AVOIDING REFORMULATIONS

The s -step (CA-Arnoldi) reformulation [15, 8, 6] computes s basis vectors $\mathbf{v}, \mathbf{A}\mathbf{v}, \dots, \mathbf{A}^s\mathbf{v}$ per communication phase. The *matrix-powers kernel* [18] computes all s products in one pass over the operator, tiling the grid so that each panel and its s -deep halo stay cache-resident—attacking the vertical wall. The block is then orthogonalized at once—here by CholeskyQR2 [23], whose certificate $\kappa(\mathbf{B}) \leq u^{-1/2}$ (with u the unit roundoff) is a floating-point theorem rather than a machine property—attacking the horizontal wall with $O(m/s)$ reductions in place of MGS’s $O(m^2)$.

Algorithm 1: Adaptive Krylov Exponential Integrator for $\mathbf{u}' = \mathbf{A}\mathbf{u} + \mathbf{b}$

Input: Matrix $\mathbf{A} \in \mathbb{R}^{N \times N}$, vector $\mathbf{b} \in \mathbb{R}^N$, initial condition $\mathbf{u}_0$, final time T , time-step $\Delta\tau$, tolerance tol

Output: Solution snapshots $\{\mathbf{u}_k\}_{k=0}^K$ at $\tau_k = k \Delta\tau$

```

1   $\tau \leftarrow 0$ ;
2  for  $k = 0, 1, \dots, K-1$  do
3       $h \leftarrow \min(\Delta\tau, T - \tau)$ ;
4       $m \leftarrow 1$ ;  $\text{err} \leftarrow \infty$ ;
5      Form the extended system  $\tilde{\mathbf{A}}, \tilde{\mathbf{v}}$  from  $\mathbf{A}, \mathbf{b}, \mathbf{u}_k$  // Eq. (3.0.2)
6      /* Adaptive Krylov loop */
7      while  $m \leq N+1$  do
8          Run  $m$  steps of Arnoldi on  $(\tilde{\mathbf{A}}, \tilde{\mathbf{v}}) \Rightarrow \mathbf{V}_{m+1}, \bar{\mathbf{H}}_m$  // Algorithm 2
9          Store  $\bar{h}_{m+1,m} \leftarrow [\bar{\mathbf{H}}_m]_{m+1,m}$  // sub-diagonal entry
10          $\mathbf{H}_m \leftarrow \bar{\mathbf{H}}_m(1:m, 1:m)$  // drop last row
11          $\beta \leftarrow \|\tilde{\mathbf{v}}\|$ ;
12          $\mathbf{f} \leftarrow e^{h\mathbf{H}_m} \mathbf{e}_1$  // small  $m \times m$  matrix exponential
13          $\mathbf{w} \leftarrow \beta \mathbf{V}_m \mathbf{f}$  // Krylov approximation in full space
14         /* A posteriori error estimate */
15          $\text{err} \leftarrow |\bar{h}_{m+1,m}| \cdot \|\mathbf{f}\|_m$ ;
16         if  $\text{err} < \text{tol}$  then
17             break;
18         end
19          $m \leftarrow m+1$ ;
20     end
21      $\mathbf{u}_{k+1} \leftarrow \mathbf{w}(1:N)$  // extract first  $N$  components
22      $\tau \leftarrow \tau + h$ ;
23 end
    
```

Algorithm 2: Arnoldi Iteration

Input: Matrix $\tilde{\mathbf{A}} \in \mathbb{R}^{(N+1) \times (N+1)}$, vector $\tilde{\mathbf{v}} \in \mathbb{R}^{N+1}$, subspace dimension m , breakdown tolerance tol

Output: Orthonormal basis $\mathbf{V}_{m+1} \in \mathbb{R}^{(N+1) \times (m+1)}$, upper Hessenberg $\bar{\mathbf{H}}_m \in \mathbb{R}^{(m+1) \times m}$

```

1   $\beta \leftarrow \|\tilde{\mathbf{v}}\|$ ;  $\mathbf{v}_1 \leftarrow \tilde{\mathbf{v}}/\beta$ ;
2  for  $j = 1, \dots, m$  do
3       $\mathbf{w} \leftarrow \tilde{\mathbf{A}} \mathbf{v}_j$ ;
4      for  $i = 1, \dots, j$  do
5          // Modified Gram-Schmidt
6           $h_{i,j} \leftarrow \mathbf{v}_i^T \mathbf{w}$ ;
7           $\mathbf{w} \leftarrow \mathbf{w} - h_{i,j} \mathbf{v}_i$ ;
8      end
9       $h_{j+1,j} \leftarrow \|\mathbf{w}\|$ ;
10     if  $h_{j+1,j} < \text{tol}$  then
11         break // Invariant subspace found (happy breakdown)
12     end
13      $\mathbf{v}_{j+1} \leftarrow \mathbf{w}/h_{j+1,j}$ ;
14 end
    
```

What the reformulation gives up is numerical robustness: the monomial basis $[\mathbf{v}, \mathbf{A}\mathbf{v}, \dots, \mathbf{A}^s\mathbf{v}]$ grows geometrically ill-conditioned in s (each vector rotates toward the dominant eigenvector), which caps the usable block width. Quantifying that cap—predicting it from the spectrum, certifying it, and measuring where the prediction survives non-normality—is a central thread of Section 8.

An alternative not pursued here is the rational Krylov subspace [4], which replaces the sequential polynomial recurrence with m independent shifted solves. Its parallelism is attractive, but each solve is itself a sparse linear system whose iterative solution re-imports both communication walls, and the near-optimal pole selection adds problem-specific tuning. The s -step route was chosen because its costs are analyzable in machine terms—which is precisely what the regime map requires.

4 EXPERIMENTAL PLATFORM AND METHODOLOGY

Every CPU result in this thesis was gathered on a single bare-metal workstation (“puffin”): an AMD Ryzen Threadripper 3960X, 24 cores in a single socket and a single NUMA node, organized as 8 *core complexes* (CCX)—clusters of 3 cores that share one 16 MiB slice of L3 cache (128 MiB total). Per-core caches are 32 KiB L1d and 512 KiB L2, nominal maximum clock is 3.8 GHz, and main memory is 4-channel DDR4-3200 with a theoretical 102.4 GB/s roof. Each core also exposes a second hardware thread via simultaneous multithreading (SMT). The machine also hosts an NVIDIA RTX 3090 (24 GB), which is one half of the GPU pair below. The machine’s derived constants live in a single header (`include/machine.hpp`) under the key `amd-3960x`, and every predicted coordinate in the regime study is computed from them.

THE GPU PLATFORM. The GPU study of Section 9 adds a second, deliberately controlled machine, “syngé”: a two-node cluster carrying a pair of NVIDIA Tesla V100-PCIE-16GB cards per node (four in total), the two cards within a node joined by PCIe across a cross-socket hop with no NVLink, and the two nodes by a 100 Gb/s Omni-Path fabric (over which the NCCL collective library selects its InfiniBand-verbs transport). Syngé’s V100 is the datacenter half of a near-controlled pair whose consumer half is puffin’s RTX 3090: the two cards share a 6 MB L2 and a ~ 820 GB/s achieved bandwidth class and differ almost only in double-precision throughput—an $11\times$ gap that the two-card experiments turn into an instrument. Their constants live in a separate header (`include/gpu_machine.hpp`), typed distinctly from the CPU constants so that a CPU value reaching a GPU coordinate is a compile error rather than a plausible wrong number, and are calibrated per card on an idle device rather than read from datasheets.

WORKSTATION SELECTION. A cloud arm was seriously considered, and its attractions are real: cloud providers offer server-grade hardware at reasonable hourly cost where puffin is commodity desktop silicon, better hardware would open larger problem configurations, and in a commercial setting these services are exactly where a desk would run its own research. It was dropped anyway, on two grounds. Contention: timing variance that looks internally consistent is simultaneously evidence of memory-bound behavior and of noise from co-tenants, and from inside the guest the two are not separable—so economic feasibility comes bundled with uninterpretable measurements. Topology, the sharper objection: the hypervisor reports a socket and a private last-level cache slice *per vCPU*, a fiction of the virtualization rather than the silicon. Both coordinates of the regime map are functions of real cache geometry (one is a cache-capacity ratio, the other is set by how many cache-domain boundaries a reduction tree crosses), so neither is knowable on an invented topology, and the map cannot be drawn there at all. The conclusion is that this study needs not merely competitive hardware but *fully controlled* hardware; the same reasoning applies on shared clusters, where other users’ large jobs contend for the same memory system. Bare metal removes the hypervisor but not OS scheduling noise,

so every timed quantity below is reported as a median with an inter-quartile range over repeated trials; on a boost-enabled chip timing noise is right-skewed, and the median reports *sustainable* throughput where the minimum would report an unsustainable boost spike.

REPRODUCIBILITY. Every number quoted in this thesis names the script that reproduces it. The repository builds with CMake and GCC 16 (C++23); sweep scripts write CSVs under `data/`, which is generated locally and excluded from version control precisely so that no quoted figure can drift from its generating run; each figure is produced by a self-contained plotting script with inline dependency metadata. The regime executables read machine constants from a single audited header, so porting the study to a new machine is a matter of deriving one new constants block.

5 BENCHMARK: REPRODUCING NIESEN–WRIGHT

Five methods are compared. Three are time-steppers over a fixed number of substeps: Crank–Nicolson ($\theta = 0.5$, fully implicit, second order, one sparse LU factorization per run), and two alternating-direction-implicit splittings—Douglas–Rachford [9], with three direction-split solves per step, and Hundsdorfer–Verwer [16, 17], a Douglas–Rachford predictor with a corrector. The remaining two exponentiate the operator. The matrix exponential (ME) forms $e^{T\tilde{\mathbf{A}}}$ in one shot by scaling and squaring [13, 1] with an ∞ -norm early exit, choosing its own substep count. The Krylov subspace exponential integrator (KSM-EI) is the method of Section 3, growing the basis until an a-posteriori estimate meets the tolerance.

The benchmark reproduces Figure 1 of Niesen and Wright [19]: log-log plots of ODE error against CPU time for all five methods, across both option types and both grid sizes (Figure 1).

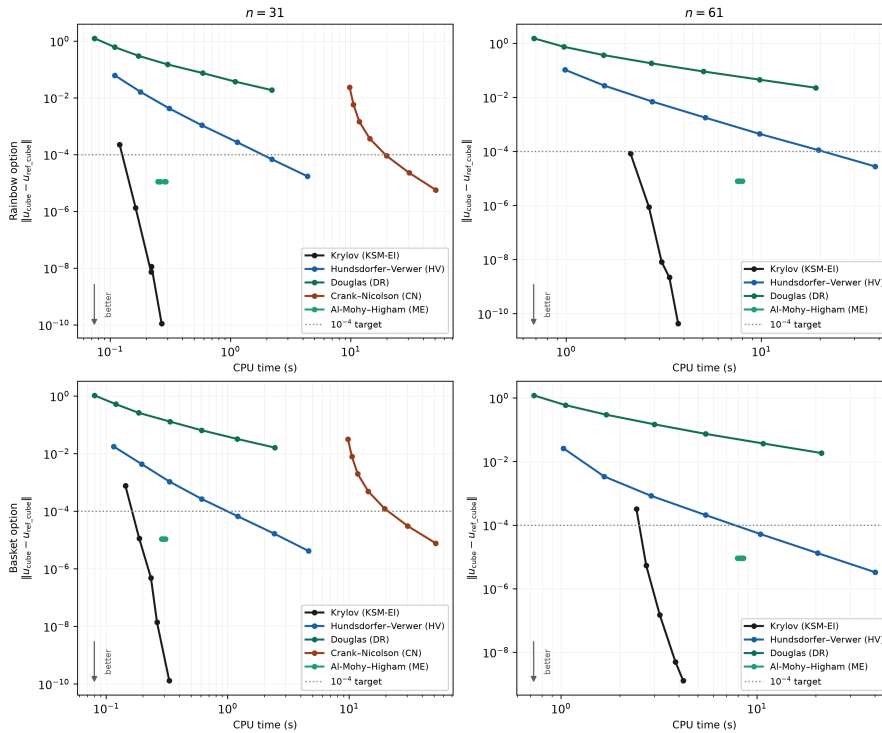


Figure 1: ODE error vs. CPU time for the five methods, across both payoffs and grid sizes, reproducing Niesen–Wright Figure 1. The Krylov integrator dominates in the $\sim 10^{-4}$ accuracy band; Crank–Nicolson’s non-monotone stiff-mode ringing ($R(-\infty) = -1$) is visible as predicted.

THE REFEREE. The measurement is against the exact ODE solution, not the analytic price. Before each option type is benchmarked, a high-accuracy matrix-exponential referee is computed at maximum Taylor degree $m = 55$ and $\theta = 9.9$, giving $s = \lceil T\|\tilde{\mathbf{A}}\|_1/\theta \rceil$ internal substeps at a tolerance of $2^{-53} \approx 1.1 \times 10^{-16}$. Scoring against the analytic price instead would fold spatial discretization error into a claim about time integration, so the referee is deliberately a different object from the contestants: the exact solution of the same semi-discrete system. The error is the Euclidean norm over the $9 \times 9 \times 9$ grid neighborhood centered on the spot rather than the value at a single node, so a lucky cancellation at one grid point cannot flatter a method.

ROLE OF THE BENCHMARK. This section is the control for everything after it: the measured method ordering matches theory, so the baseline is trusted. At small grids the one-shot matrix exponential is genuinely hard to beat—at $n = 15$ every method is pinned at the spatial error (0.066), so time-exactness buys nothing. The Krylov integrator’s advantages (adaptivity, no step-count trial-and-error, a retained trajectory) only pay at the production resolutions where the ~ 700 s cost lives.

6 PROFILE AND ROOFLINE

At $n = 61$ (226,981 unknowns, 4.23M nonzeros, ≈ 49 MB in compressed sparse column form) over 100 steps, the KSM-EI solve is overwhelmingly SpMV and Gram–Schmidt; the dense $\exp(\mathbf{H}_m)$ is marginal (Table 3). Gram–Schmidt’s share grows with the grid because the Krylov dimension m grows and MGS is quadratic in m . The *other* column collects everything outside the three instrumented kernels: vector updates, norm computations for the error estimate, and per-step bookkeeping.

Table 3: Kernel shares of KSM-EI solve time (basket, tolerance 10^{-8}). $\bar{m}$ is the mean Krylov dimension; *other* is uninstrumented work (vector updates, error estimation, bookkeeping).

Grid	$\bar{m}$	SpMV	Gram–Schmidt	dense expm	other
$n = 31$	7.60	82.9%	11.7%	0.40%	4.9%
$n = 53$	8.42	77.7%	16.8%	0.34%	5.1%
$n = 65$	9.24	76.0%	18.0%	0.21%	5.9%
$n = 73$	9.89	75.2%	18.8%	0.15%	5.9%
$n = 89$	11.34	70.3%	23.8%	0.08%	5.8%

Both kernels sit far below the compute ceiling. The cache-aware roofline [22] shows why, and where the CA opportunity is (Figure 2, Table 4). Single-core roofs on puffin, all measured: DRAM 20.9 GB/s, L3 58.2 GB/s, L2 88.6 GB/s, compute peak 68.1 GFLOP/s (an FMA loop).

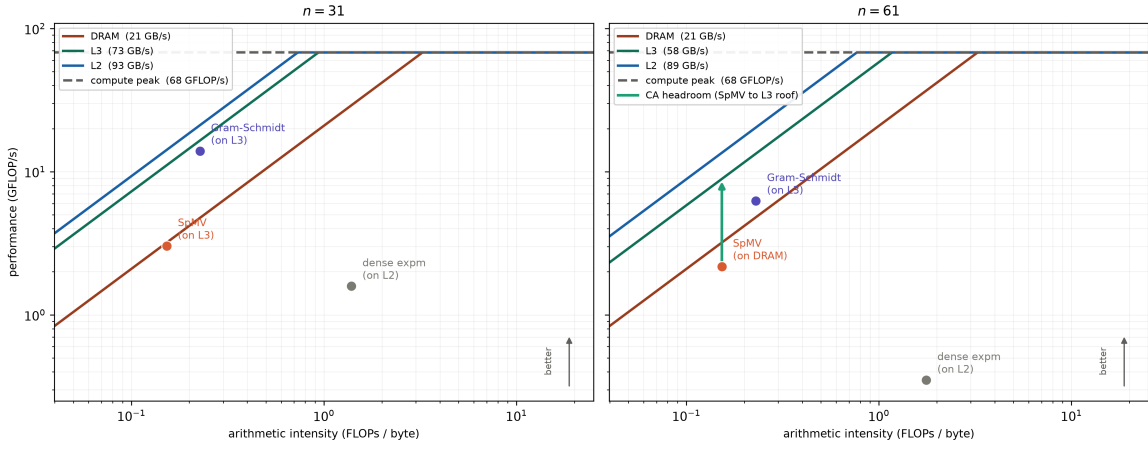


Figure 2: Cache-aware roofline for the two hot kernels at $n = 31$ and $n = 61$. At $n = 61$ the SpMV has spilled to DRAM, the vertical distance from the measured point to the L3 roof at the same arithmetic intensity is exactly the traffic a matrix-powers reformulation would recover.

Table 4: Measured kernel positions on the roofline. AI is arithmetic intensity.

Grid	Kernel	AI (FLOP/B)	Achieved (GFLOP/s)	Resident tier
$n = 31$	SpMV	0.153	3.02	L3
$n = 31$	Gram-Schmidt	0.227	13.91	L3
$n = 61$	SpMV	0.153	2.17	DRAM
$n = 61$	Gram-Schmidt	0.230	6.24	L3

The attainable gain from moving a kernel one tier up the hierarchy is

$$\text{gain} = \frac{\min(BW_{\text{fast}} \cdot AI, \text{peak})}{\min(BW_{\text{slow}} \cdot AI, \text{peak})}, \quad (6.0.1)$$

which collapses to the bandwidth ratio only while *both* roofs are still sloped. It must bend down once the fast roof reaches the compute ceiling—which the $s \times$ -higher arithmetic intensity of a matrix-powers kernel can actually reach. Quoting a flat bandwidth ratio as “the CA speedup” overstates the ceiling.

7 THE OPENMP SCALING STUDY

The scaling study measures how the two hot kernels of one Arnoldi cycle respond to added cores—it does not measure solution accuracy. The thesis of the experiment is that the two kernels fail to scale for two *different* reasons, each motivating a distinct CA mechanism (Table 2), this split is what the regime analysis of Section 8 later turns into two independent dimensionless axes.

7.1 INSTRUMENT DESIGN

To make both ceilings visible and attributable, the harness freezes the Krylov dimension at $m = 8$ with the convergence check disabled, so the Gram-Schmidt/SpMV work ratio does not drift and any runtime change across the core count P is attributable to hardware rather than a changing algorithm (the genuine $m(n)$ growth is measured separately). The SpMV stores the matrix in compressed sparse row (CSR) format—each row’s nonzeros held contiguously—and is hand-rolled, i.e. written for this

study rather than taken from a library, so that the row partition and the reduction strategy are fully under the experiment’s control: each thread is the sole writer of a contiguous, disjoint slice of the output vector, with partition seams aligned to the cache line. The harness runs MGS with parallel global reductions, exposing the synchronization cost directly, and reports per-kernel timings so a single sweep yields SpMV’s plateau and Gram–Schmidt’s roll-off on one axis. Runs pin threads with `OMP_PLACES=cores` and `OMP_PROC_BIND=close`, so each CCX (Section 4) fills before the next engages and per-CCX L3 bandwidth steps appear as discrete risers. The swept core counts are CCX-boundary aligned, and SMT lanes are excluded so no core is timed while sharing its pipelines with a second thread. Each point is the median of five to seven timed repeats after a warm-up, with the inter-quartile range as the error bar.

Two initialization arms form a controlled A/B experiment on memory placement: arm A fills the matrix from thread 0 (first-touch parks it on one memory placement), arm B has each thread first-touch exactly the row partition it will later compute on. The two arms turn out to be indistinguishable—for a reason that is itself the finding, taken up in Section 7.3. Figures 3 and 4 show the strong- and weak-scaling deliverables.

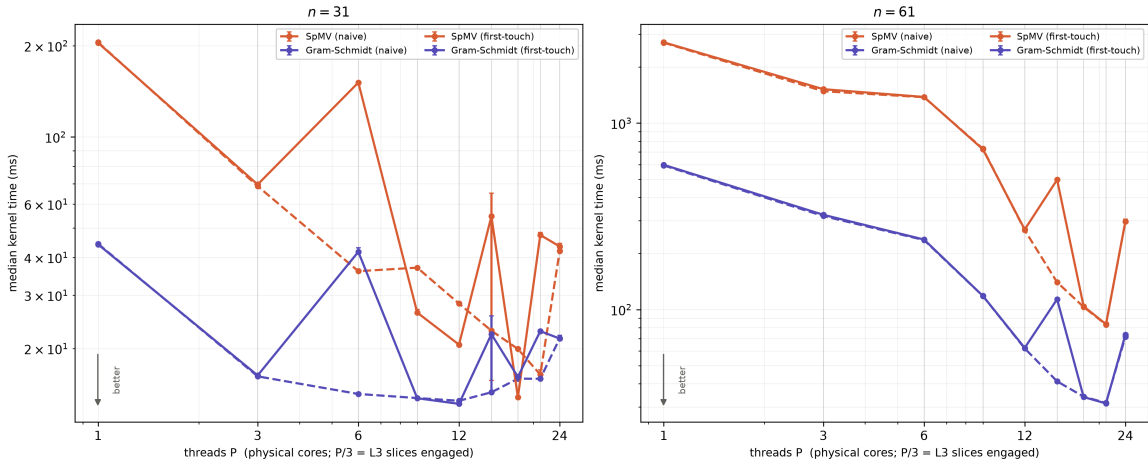


Figure 3: Per-kernel strong scaling at $n = 31$ and $n = 61$, both initialization arms. SpMV shows a plateau at $n = 31$ and a superlinear transition at $n = 61$; Gram–Schmidt shows the synchronization staircase.

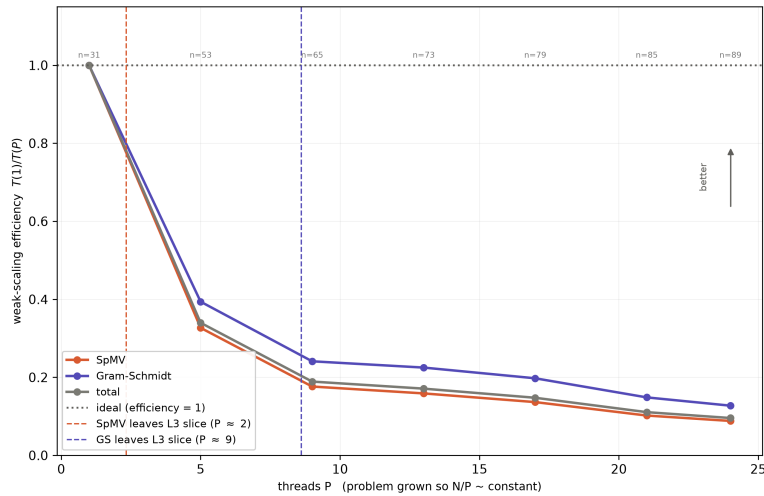


Figure 4: Per-kernel weak-scaling efficiency $T(1)/T(P)$ with $n(P) = n_1 P^{1/3}$ rounded odd, cache-tier crossings annotated.

7.2 THE QUARANTINED $m(n)$ GROWTH

The harness freezes m so that the scaling curves measure hardware; the genuine growth of the Krylov dimension with the grid is measured separately and reported as an independent numerical-analysis finding (Figure 5). The mean $\bar{m}$ per step grows from 7.6 at $n = 31$ to 11.3 at $n = 89$ —slow growth, but enough to drive Gram–Schmidt’s quadratic share upward (Table 3), and the input to the small- m safety argument of Section 8.4.

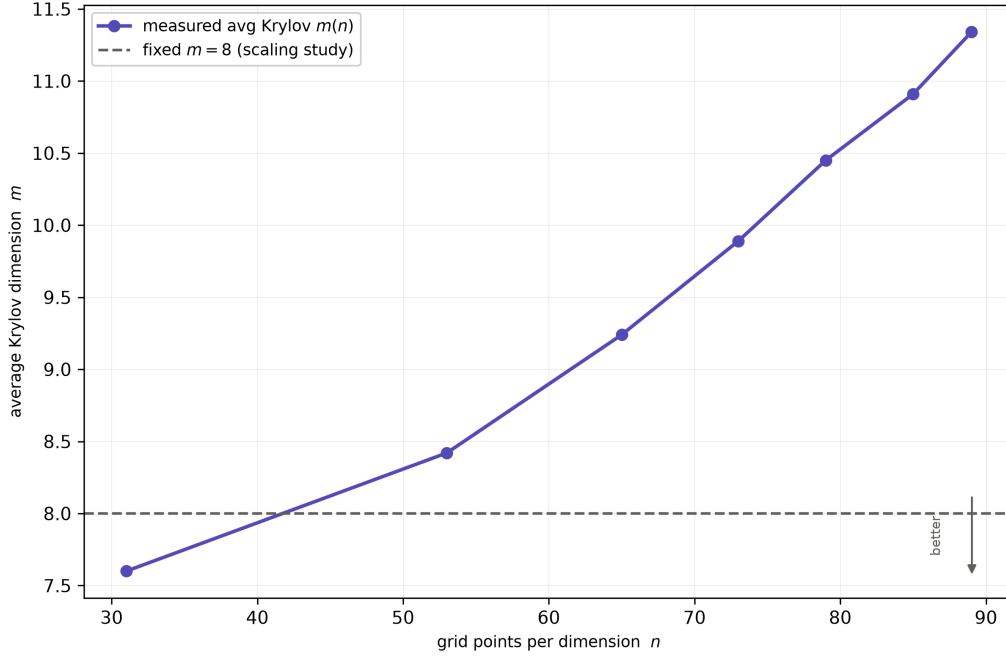


Figure 5: The $m(n)$ Krylov-growth curve, measured with the convergence check enabled and quarantined from the scaling instrument, which pins $m = 8$.

7.3 THE LOCALITY CONTROL

At $n = 61$ the SpMV runtime drops *superlinearly* ($36.7\times$ on 24 cores) with a knee at $P = 12$ (Figure 3). The candidate explanation is cache residency: once each CCX’s share of the matrix ($3 \times 52.8/P$ MiB) fits in its own 16 MiB L3 slice and stays resident across the repeated SpMV, reads come from L3 instead of DRAM. Quantitatively this checks out—the implied aggregate bandwidth crosses the socket DRAM ceiling (~ 95 GB/s) exactly at $P = 12$, which is impossible from DRAM and therefore proves L3 residency.

The initialization arms *cannot* test this: the 3960X is single-NUMA, and Zen 2 L3 is a per-CCX victim cache, so L3 residency is set by which core *accesses* a row, not by which thread first-touched the page. Both arms use the same row partition and produce identical curves. The real controlled experiment is over the *compute schedule*, which changes only which rows each core reads (Table 5, Figure 6).

Table 5: The locality control at $n = 61$: observed SpMV behavior per compute schedule (speedup at $P = 24$; implied aggregate bandwidth at $P = 12$).

Schedule	What it does	Observed behavior
block	contiguous band per core, fixed across SpMV	36.7 $\times$, knee at $P = 12$, 176 GB/s
cyclic	cache-line blocks dealt round-robin, fixed	7.0 $\times$, no knee, 36 GB/s
rotate	contiguous bands, rotating by one each SpMV	6.6 $\times$, no knee, 66 GB/s

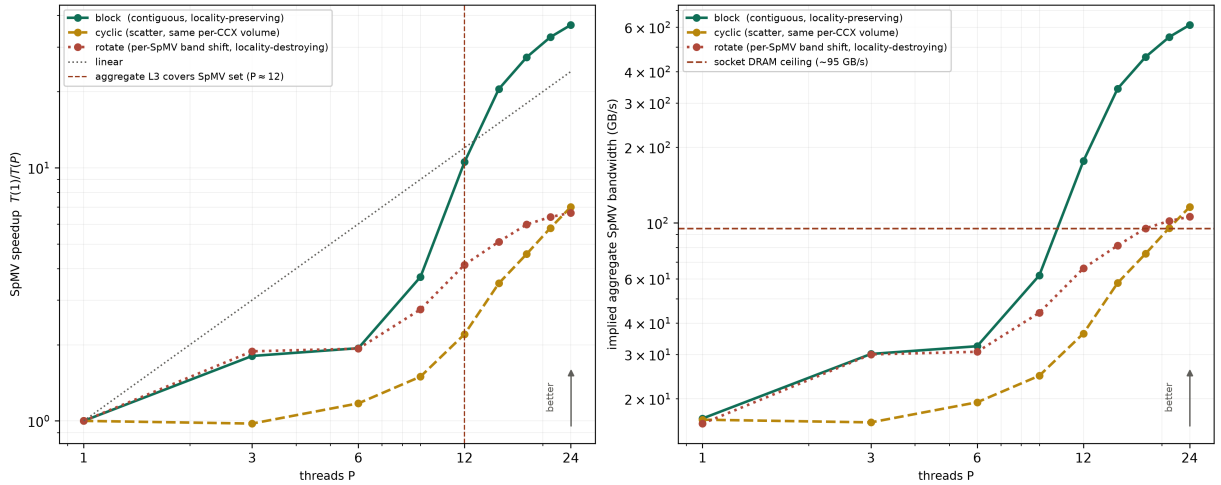


Figure 6: The locality control. Only **block** carries an aggregate bandwidth DRAM cannot supply, so the transition is genuinely L3 residency, not a scheduling coincidence.

The **cyclic** schedule was predicted to keep the transition, since it leaves per-CCX byte *volume* unchanged; it lost the transition instead, collapsing to the **rotate** curve (7.0 $\times$ vs. 6.6 $\times$, against **block**’s 36.7 $\times$). **Contiguity, not volume, is what keeps a slice resident**—exactly the property a matrix-powers tiling exploits, and exactly the property the scattered arm of the vertical sweep (Section 8.9) destroys on purpose. The identical arm-A/arm-B result is thereby the control that establishes the transition as a locality effect—the “before” that a matrix-powers reformulation would improve on.

8 REGIME ANALYSIS

The scaling study says the baseline solver is memory-bound and synchronization-bound. It does not say whether a CA reformulation would *help*, or where. The regime analysis answers that with a dimensionless map whose two axes are the two mechanisms, with a predicted crossover at 1 on each, plus a synthetic instrument that can be dialed to any point on the map and a real Black–Scholes operator that is *placed* on it by measurement.

The synthetic instrument generalizes the discretized operator of Section 2 into a family with independently controllable knobs. Its backbone is a *Kronecker sum*: one tridiagonal advection–diffusion operator $\mathbf{T} \in \mathbb{R}^{n_1 \times n_1}$ (the same three-point stencil as Section 2, with diffusion scale σ and advection

strength γ) is extended to the d -dimensional grid by

$$\mathbf{A}_d = \mathbf{T} \oplus \cdots \oplus \mathbf{T} = \sum_{i=1}^d \mathbf{I} \otimes \cdots \otimes \underbrace{\mathbf{T}}_{i\text{-th factor}} \otimes \cdots \otimes \mathbf{I} \in \mathbb{R}^{n_1^d \times n_1^d},$$

which is exactly how a separable PDE discretization acts: the same one-dimensional stencil applied along each axis in turn [12]. Two optional terms then break that clean structure on demand, mirroring the two ways the real operator of (2.0.1) departs from a textbook Laplacian. The *correlation cross term* adds $c(\mathbf{D} \otimes \mathbf{D})$, the discrete mixed second derivative coupling one axis pair—the role the $\rho_{dd'}$ terms play in the PDE. It is optional because separability is itself a variable under study: with the term off, the operator factors axis by axis and its spectrum is known in closed form; with it on, that factorization is broken in a controlled way. The *symmetric scatter permutation* relabels the grid unknowns, replacing $\mathbf{A}$ by $\mathbf{PAP}^\top$ for a permutation matrix $\mathbf{P}$: the same operator stored in a different memory layout, which destroys the banded structure a cache-blocked kernel relies on while provably changing no numerical property (gate C2 below).

The knobs are chosen so that each axis of the map moves *independently*: the grid size $N = n_1^d$ sets the Arnoldi working set and therefore R_v ; the spectrum controls $(\sigma, \mu, \gamma, \rho)$ set the basis conditioning and therefore the certified block width, without touching the memory footprint; and the scatter knob changes the memory-access pattern at fixed spectrum. One operator family can therefore visit any point on the map while holding the other coordinate fixed—which is what makes it an instrument rather than just a test matrix.

8.1 THE COORDINATES

Table 6: The two dimensionless coordinates.

Axis	Definition	Mechanism it governs	Scales like
R_v	Arnoldi working set / aggregate last-level cache	vertical: matrix-powers tiling, avoiding DRAM traffic	$\sim 1/P$
R_h	reduction cost / compute per core between reductions	horizontal: s -step, cutting the reduction count	$\sim P \log P$

Each coordinate is built as demand over supply. R_v divides the bytes the Arnoldi working set occupies by the bytes of last-level cache available to hold it; R_h divides the time a global reduction costs by the compute each core performs between reductions. A ratio of that shape has one distinguished value: at $R = 1$, supply exactly covers demand. Below it the working set already fits in cache (or the reduction is already cheap relative to compute), so the corresponding CA mechanism has nothing to recover; above it there is genuine traffic or synchronization for the mechanism to reclaim. The thresholds θ_v and θ_h name these predicted crossover values of R_v and R_h , and both equal 1 by construction—which is also what makes them portable, since a dimensionless ratio crossing 1 means the same thing on any machine. The two thresholds cut the map into four corners: neither mechanism pays (lower-left), horizontal only (lower-right), vertical only (upper-left), and both (upper-right). The Krylov dimension m was demoted from an axis to a *modulator*: it moves working sets, so it moves R_v , but it is not itself a portable coordinate—two machines with the same m can sit on opposite sides of θ_v .

R_h 's numerator needs a magnitude, not just a shape in P , or θ_h cannot be drawn at all. A calibration instrument measures it directly as a bare scalar all-reduce over an empty team—no payload, no application—and the model fitted to it is

$$t_{\text{reduce}}(P) = t_{\text{intra}} \cdot \text{intra levels}(P) + t_{\text{cross}} \cdot \min(\text{cross levels}(P), L^*). \quad (8.1.1)$$

On puffin: $t_{\text{intra}} = 120.8 \text{ ns}$, $t_{\text{cross}} = 652.8 \text{ ns}$ (a cross-CCX tree level costs $5.4\times$ an intra-CCX one), and $L^* = 2.20$ —past roughly two cache-domain crossings, subtree concurrency hides the rest. Measured tree all-reduce latency runs from 9.6 ns at $P = 1$ to $1.70 \mu\text{s}$ at $P = 24$ ($R^2 = 0.98$ against the model). The cost is set by how many cache-domain boundaries the tree crosses, not by core count—which is precisely why this coordinate cannot be measured on a virtualized topology (Section 4).

8.2 THE INSTRUMENT AND ITS GATES

Before any timed experiment, a single-threaded control program verifies the instrument’s arithmetic claims (Table 7). Nothing in it is timed and no result depends on the host, so its verdicts are properties of the operator, not the machine. Each control is either a *gate*—a claim the rest of the study assumes, whose failure invalidates everything downstream—or a *finding*, a measured result reported in its own right.

Table 7: The controls. A gate failure is fatal; a finding is a result.

Control	Claim under test	Status
C1	the Kronecker-sum spectrum is analytic and correct	gate
C2	the scatter knob is an exact similarity transform (so a pure- R_v dial exists)	gate
C3	measured m obeys the Hochbruck–Lubich spectral bound [14]	gate (Hermitian PSD)
C3b	a spectral shift leaves m fixed	finding
C4	κ of the matrix-powers basis equals the row-scaled Vandermonde from the spectrum	gate when normal
C5	the CholeskyQR certificate $\kappa \leq u^{-1/2}$ [23] is sound	gate
C5b	worst-case block count over the physical ensemble (“small- m safety”)	finding
C6	where the operator is non-normal, does the spectral prediction survive?	finding

C2 is load-bearing. The scatter knob applies a symmetric permutation $\mathbf{PAP}^\top$ with $\mathbf{P}$ a permutation matrix; since $\mathbf{P}$ is orthogonal, $\mathbf{PAP}^\top = \mathbf{PAP}^{-1}$ is a similarity transform, and similar matrices share their spectrum exactly [12, 21]. The permutation therefore moves bytes—which rows sit near which—and provably nothing else. If it moved the spectrum, the “pure R_v line” would be an illusion and every vertical result would be confounded. The sweep runs in five phases—gate, confound, m -range, non-normal, and finally the real discretized Black–Scholes operator with its actual discounted payoff, swept across grids. That last phase is *transfer by measurement*: it lands the real operator on the same (R_v, R_h) plane as the synthetic instrument.

8.3 BASIS CONDITIONING AND THE CONFOUND CONTROL

The certified s -step block width s_{max} is predicted from the spectrum alone (a row-scaled Vandermonde [21]), then measured. Where a closed-form spectrum exists the prediction is **integer-exact on 20 of 21 runs**; the single exception is the most violently non-normal point in the sweep ($\gamma = 0.4$, $\kappa(X) = 7.7 \times 10^8$) and is off by one step. Every deviation larger than that lives in the variable-advection arm—the family the log-robustness relation (Section 8.5) explicitly excludes. The real Black–Scholes operator deviates by at most one step at any grid measured (Figure 7).

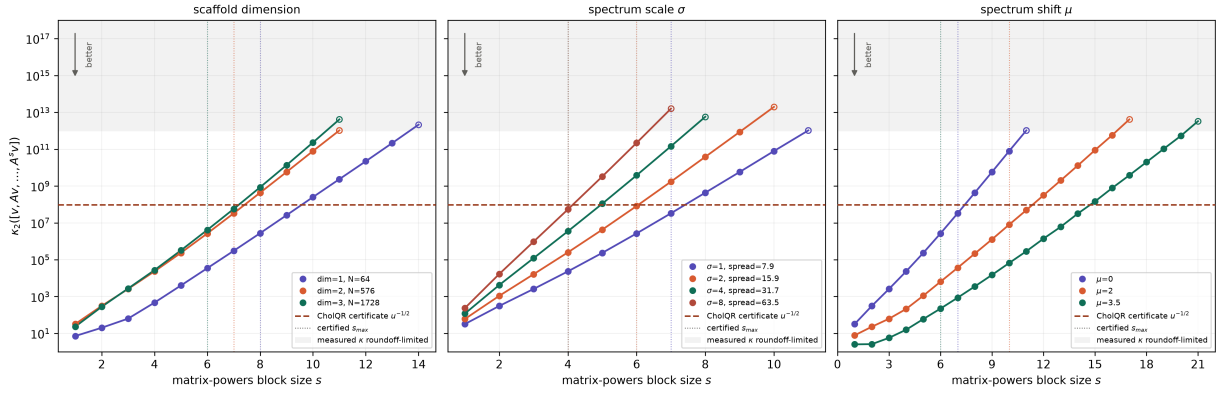


Figure 7: Predicted vs. measured basis condition number and the certified block width $s_{\max}$ across the swept operator family.

The obvious confound is that $s_{\max}$ might just be tracking the Krylov dimension m . The algebra says it should not: for a normal operator $\mathbf{A} = \mathbf{X}\mathbf{\Lambda}\mathbf{X}^*$, the matrix-powers block factors as

$$[\mathbf{v}, \mathbf{A}\mathbf{v}, \dots, \mathbf{A}^s \mathbf{v}] = \mathbf{X} \text{diag}(\mathbf{X}^* \mathbf{v}) \mathbf{V}_s(\lambda_1, \dots, \lambda_N),$$

where $\mathbf{V}_s$ is the Vandermonde matrix in the eigenvalues—so the block’s conditioning is governed by the eigenvalues and the starting vector’s eigen-coefficients, and m appears nowhere [21, 6]. The experiment confirms it (Figure 8): with the spectrum pinned and m moved through 5–12 (via the step size and tolerance), $s_{\max}$ stays flat at 7; with m pinned at 5 and the spectrum scaled and shifted, $s_{\max}$ ranges from 4 to 14. Safety is a property of the spectrum, not of the Krylov dimension.

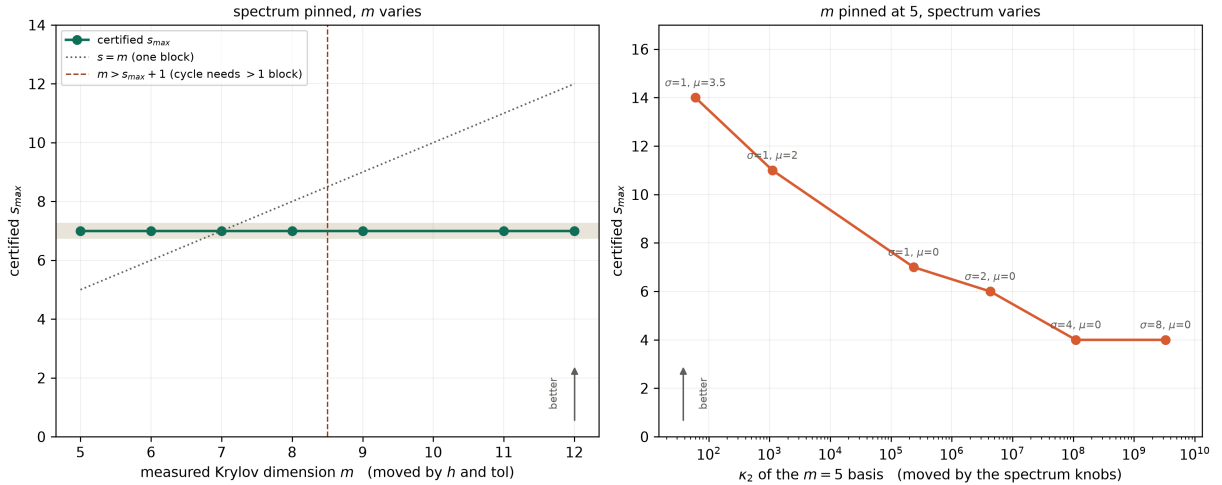


Figure 8: The confound control: $s_{\max}$ under pinned spectrum with varying m (flat), and pinned m with varying spectrum (4–14).

8.4 THE REDUCTION LEDGER AND SMALL- m SAFETY

The comparison is MGS against CholeskyQR2 because those are the two orthogonalizers actually in contention: MGS is what the baseline integrator runs (Algorithm 2), so it sets the synchronization count to beat, and CholeskyQR2 is the block orthogonalizer the s -step arm adopts. CholeskyQR2 was chosen over the alternatives on two grounds: it orthogonalizes an entire block in two global reductions regardless of block width, and it carries the explicit stability certificate $\kappa(\mathbf{B}) \leq u^{-1/2}$ [23]. Plain CholeskyQR halves the reduction count but loses the certificate; TSQR reaches a comparable count through a reduction tree at markedly higher implementation complexity [8, 15].

The *reduction ledger* (Table 8) is the resulting bookkeeping: for each Krylov dimension m , the number of global reductions one Arnoldi cycle performs under each scheme. MGS pays $1 + m(m + 3)/2$; the s -step arm pays $1 + 2\lceil m/(s_{\max} + 1) \rceil$. Across the real problem’s measured window $m = 8$ – 11 , at the $s_{\max} = 7$ certified for this spectrum, that is an **11–16 $\times$ cut in reduction count**. The ledger and the certificate are linked, because the cut is only collectible while every block stays certified: a wider block cuts more reductions but conditions worse, so the certified $s_{\max}$ is exactly the widest cut the ledger may claim. This is where *small- m safety* closes the argument. The real operator’s Krylov dimension is small and grows slowly ($m = 6$ – 8 across the eigensolvable grids $n = 10$ – 20 , and 7.6 to 11.3 across $n = 31$ – 89 in production), and a certified block yields $s_{\max} + 1$ vectors, so an entire Arnoldi cycle fits in **2–3 certified blocks**: the full cut is collected while the machinery never runs deep enough for monomial-basis conditioning to become the dominant risk.

Table 8: The reduction ledger at $s_{\max} = 7$.

m	MGS reductions	s -step + CholQR2	cut
5	21	3	7.0 $\times$
8	45	3	15.0 $\times$
9	55	5	11.0 $\times$
11	78	5	15.6 $\times$
12	91	5	18.2 $\times$

8.5 NON-NORMALITY: THE LOG-ROBUSTNESS RELATION

The spectral prediction of $s_{\max}$ assumes normality, and the real operator is not normal (mixed-derivative cross terms plus convection). The prediction error is measured as $|\log_{10} \kappa_{\text{meas}} - \log_{10} \kappa_{\text{pred}}|$, in *decades* (factors of ten), because the certificate consumes κ only through its logarithm: the threshold $u^{-1/2}$ is a log-scale cutoff, and each extra power appended to $[\mathbf{v}, \mathbf{A}\mathbf{v}, \dots, \mathbf{A}^s \mathbf{v}]$ multiplies the block’s condition number by a roughly constant factor, measured at a median of 1.05 decades per power over the swept family (inter-quartile range 0.95 to 1.12). A factor-of-ten miss therefore matters equally anywhere on the axis, and one s -step corresponds to 1.2 decades: an error below that line cannot move the certified block width by a full step. The rigorous two-sided bound is that this error is at most $\log_{10} \kappa(X)$, the eigenvector conditioning.

Measured, the data sits far below the rigorous bound (Figure 9). The fitted slope of (error in decades) against $\log_{10} \kappa(X)$ reads as a degradation rate—how many decades of prediction error each decade of eigenvector conditioning costs. For the constant-coefficient mechanisms (advection, correlation, and their combination) the slope is **0.069**: $\kappa(X)$ would have to grow by roughly 17 decades before the prediction slipped a full s -step. For variable-coefficient advection (a position-dependent ramp) it is **0.250**, about $4\times$ steeper: five decades suffice. The factor-of-four gap scopes the fitted relation to the *constant-coefficient family*; it is not mechanism-independent, which is exactly why the variable-advection knob exists in the instrument. (Both fits use the canonical $\kappa(X)$ axis of Section 8.6; the dense axis is degenerate on part of this family and would shift the slope immaterially, to 0.061.)

The real Black–Scholes operator is placed on this figure as a test point, excluded from every fit. In log-price coordinates its coefficients are constant, so it should—and does—behave like the shallow family: its prediction error runs 0.19 to 0.85 decades at $\kappa(X)$ up to 2.3×10^3 , comfortably **under one s -step** at every grid measured. Had it tracked the steeper trend instead, that would have flagged a variable-coefficient-like mechanism hiding in the real operator, and the certificate’s usable range would shrink four-fold in decades.

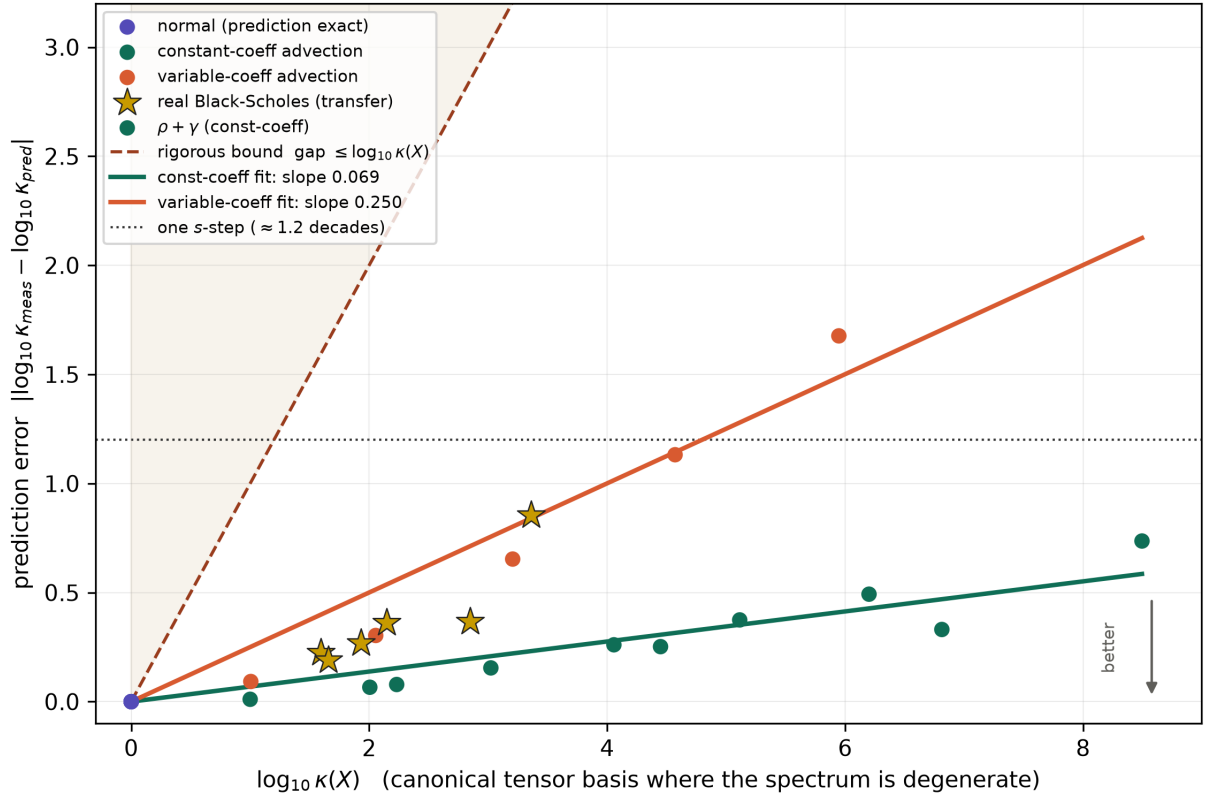


Figure 9: The log-robustness relation: prediction error in decades vs. $\log_{10} \kappa(X)$. The rigorous bound is the identity line; the measured data grows logarithmically, with the constant-coefficient and variable-coefficient families fitted separately. The real Black-Scholes operator (test point, excluded from fits) sits under one s -step at every grid.

Operator by operator, what the spectral prediction needs is normality; definiteness and separability are conveniences (Table 9).

Table 9: Which structural properties the spectral prediction of $s_{\max}$ actually needs.

Operator	Normal?	Definite?	Separable?	Spectral prediction
Laplacian	yes	PSD	yes	exact (closed form)
shift μ	yes	indefinite	yes	exact ($\kappa(X) = 1$)
correlation ρ	yes	PSD	no	exact (dense eigensolve)
constant γ advection	no	—	yes	survives (< 1 step)
$\rho + \gamma$	no	—	no	survives (< 1 step)
variable γ advection	no	—	no	must be measured (> 1 step)

The rows are cumulative modifications of one backbone. The Laplacian is the pure-diffusion Kronecker sum of Section 8, whose eigenvalues are the closed-form tridiagonal-Toeplitz values of Section 8.7 summed across axes. The shift row adds $\mu \mathbf{I}$, which translates every eigenvalue and leaves the eigenvectors untouched. The correlation row adds the cross term $c(\mathbf{D} \otimes \mathbf{D})$; the advection rows add the skew-symmetric first difference, with strength γ either constant or ramped with position. The three column properties are standard [21, 12]: an operator is *normal* when $\mathbf{A}\mathbf{A}^\top = \mathbf{A}^\top\mathbf{A}$, equivalently when it has an orthonormal eigenbasis ($\kappa(X) = 1$); *definite* (here, positive semi-definite, PSD) when it is symmetric with all eigenvalues on one side of zero; and *separable* when it factors as a Kronecker sum, so its spectrum is the set of sums of 1D eigenvalues. Normality is what the Vandermonde factorization of Section 8.3 needs to hold exactly—without it the factorization acquires the eigenvector matrix as a

distortion, bounded only through $\kappa(X)$. Separability merely decides *where the eigenvalues come from*: a closed form when separable, a dense eigensolve when not, either of which feeds the same prediction. The last column then reads: *exact* means the predicted $s_{\max}$ matches the measured one integer-exactly; *survives* means normality is broken but the prediction stays within one step; *must be measured* means it does not, and only measurement certifies the block width.

8.6 CONDITIONING GROWTH WITH ASSET DIMENSION

The remaining question for the certificate is how $\kappa(X)$ grows with the number of assets d . This axis matters because the application grows the asset count, not the grid. A refinement study swept the grid at fixed $d = 3$ and found $\kappa(X)$ flat, which the *mesh Péclet number* explains: the per-cell ratio of advection to diffusion, $\text{Pe}_h = \gamma \Delta x / (2\nu)$, is the dimensionless quantity that governs how strongly convection distorts a discrete convection–diffusion operator [16]. Here $\log \kappa(X) \sim d n_1 \gamma$ with $\gamma \sim C/n_1$, so the grid factor cancels—refinement leaves $\kappa(X)$ unchanged, and only d remains as a growth axis.

The growth is linear in $\log_{10} \kappa(X)$ against d , with a slope known in closed form. The synthetic operator applies its cross term to exactly one axis pair at every d , so it factors as a Kronecker sum $\mathbf{A}_d = \mathbf{B}_{01} \oplus \mathbf{T} \oplus \cdots \oplus \mathbf{T}$, where $\mathbf{B}_{01}$ is the coupled two-axis block and $\mathbf{T}$ the one-dimensional factor. The eigenvector matrix of a Kronecker sum is the Kronecker product of the factors’ eigenvector matrices, and the singular values of a Kronecker product are all pairwise products of the factors’ singular values [12]—so condition numbers multiply:

$$\kappa(X) = \kappa(X_{01}) \kappa(X_1)^{d-2} \implies \log_{10} \kappa(X) = 0.40499 d + 0.01422. \quad (8.6.1)$$

“Closed form” means precisely this: the slope $0.404995 = \log_{10} \kappa(X_1)$ is computed from one $n_1 \times n_1$ eigensolve of the 1D factor, not fitted to data along d . The observed $d = 2 \rightarrow 3$ increment is 0.40500, and the formula reproduces both degeneracy-free points to 3×10^{-7} .

The dense $\kappa(X)$ that a straightforward eigensolve returns must *not* be fitted along d , because from $d = 4$ it is not a property of the operator at all. The reason is spectral *degeneracy*: a spectrum is called degenerate when some eigenvalue is repeated (has multiplicity greater than one), so that its eigenvectors span a subspace of dimension larger than one instead of a single line; a spectrum with every eigenvalue distinct is called *simple*. Inside a repeated eigenspace, every orthonormal basis is an equally valid set of eigenvectors—so “the” eigenvector matrix, and with it the dense $\kappa(X)$, is no longer uniquely defined [12]. That is exactly what happens here: axes $2, \dots, d-1$ carry no cross term, so interchanging any two of them is an exact symmetry of $\mathbf{A}$, the spectrum repeats, and the value reported is whichever admissible basis the eigensolver’s rounding happened to produce. A symmetric permutation $\mathbf{PAP}^\top$ is an exact similarity, so any function of the operator is invariant under it—yet the dense value moves by a factor of 3.84 at $d = 4$ and 7.76 at $d = 5$ across eight permutations once the spectrum is degenerate. The study therefore reports a canonical, basis-independent value (`kappa_X_struct`, computed per Kronecker factor) alongside the dense one (Table 10, Figure 10).

Table 10: Canonical vs. dense $\kappa(X)$ along the asset dimension.

d	N	canonical $\kappa(X)$	dense $\kappa(X)$	spectrum
2	16	6.671	6.671	simple (agree to 3×10^{-7})
3	64	16.95	16.95	simple (agree to 3×10^{-7})
4	256	43.07	109.5	degenerate: dense meaningless
5	1024	109.4	5.2×10^3	degenerate: dense meaningless
6	4096	278.1	3.7×10^4	degenerate: dense meaningless

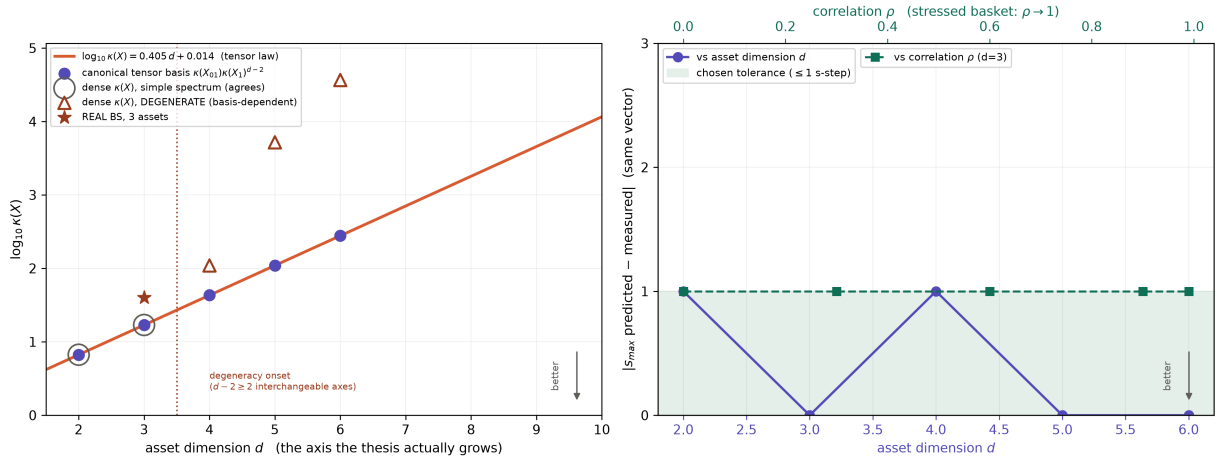


Figure 10: Conditioning growth with asset dimension: canonical $\kappa(X)$ grows geometrically in d with an analytic slope; the dense value diverges from it exactly where the spectrum degenerates.

The same degeneracy appears at one point of the correlation sweep ($n_1 = 8$, $d = 3$), and the canonical value resolves it. For every $\rho > 0$ the cross term couples one axis pair, the spectrum stays simple, and the dense and canonical values agree exactly. At $\rho = 0$ the cross term vanishes, the operator becomes a pure Kronecker sum with all three axes interchangeable, and the dense value is degenerate: of the $N = n_1^d = 8^3 = 512$ eigenvalues, only 89 are distinct. Read on the canonical anchor (Table 11), **correlation raises $\kappa(X)$ monotonically**, from 688 at $\rho = 0$ to 3405 at $\rho = 0.99$ —about $5\times$ across the full range.

Table 11: The correlation arm ($n_1 = 8$, $d = 3$). At $\rho > 0$ the dense value matches the canonical one exactly; at $\rho = 0$ the operator is a pure Kronecker sum and the dense value is degenerate.

ρ	0	0.3	0.6	0.9	0.99
canonical $\kappa(X)$	688.0	851.8	1566.9	2277.9	3405.2
dense $\kappa(X)$	<i>11350</i>	851.8	1566.9	2277.9	3405.2

Dimension still dominates: ρ across its entire admissible range buys about 1.7 assets' worth of conditioning, and $\rho \leq 1$ while d is unbounded—but correlation never helps. The practical consequence is that a-priori conditioning bounds age quickly along d : because $\kappa(X)$ grows geometrically in d while staying flat under grid refinement, a certified block width carried over from low asset counts loses its safety margin at higher ones, and at high d the certification has to come from measurement. The identity's scope is the synthetic instrument: the real basket operator couples all three asset pairs, so the tensor identity, which rests on one coupled pair, does not extend to it.

8.7 THE COMPUTABILITY GAP

The certificate is validated by dense eigensolve, which caps the control at $N \leq 32,768$, while the performance claims live at $N \sim 10^5$. A natural bridge suggests itself: each one-dimensional factor of the operator is *Toeplitz* (constant along every diagonal), and the eigenvalues of a tridiagonal Toeplitz matrix are known in closed form, $\lambda_k = a + 2\sqrt{bc} \cos(k\pi/(n+1))$ [12]—so an analytic spectrum could replace the eigensolve entirely.

The bridge fails for this operator, and the obstruction is identifiable. The cross term factors exactly as $c(\mathbf{D} \otimes \mathbf{D})$ with $\mathbf{D} = \text{tridiag}(-1, 0, +1)$ (verified to machine zero), and a diagonal similarity $\text{diag}(z^j)$

symmetrizes $\text{tridiag}(b, a, c)$ iff $z^2 = c/b$. The two terms demand incompatible z on the same axis: $z^2 = (1 - \gamma)/(1 + \gamma)$ for advection–diffusion against $z^2 = -1$ for the cross factor. Numerically, the sine basis leaves 3.7% off-diagonal mass with the cross term against 2.8×10^{-14} without, no single z helps, and the obstruction persists at $\gamma = 0$ —so it is the Dirichlet boundary condition, not the advection. What survives is the tensor identity (8.6.1), which gives $\kappa(X)$ at any d from a single $n_1^2 \times n_1^2$ eigensolve, with no cap—for the *synthetic operator*. The gap remains open for the real Black–Scholes operator, whose all-pairs correlation breaks the Kronecker factorization.

8.8 THE HORIZONTAL MECHANISM

Cutting the reduction count 11–16 $\times$ leaves Gram–Schmidt wall-clock unchanged. Two independent measurements explain why, and they agree:

1. **The primitive was never the bottleneck.** Replacing the naive barrier-plus- $O(P)$ -scan reduction with a proper binary tree—a 1.7 $\times$ improvement in the bare all-reduce latency at $P = 24$ ($2.91 \mu\text{s}$ down to $1.70 \mu\text{s}$)—leaves Gram–Schmidt wall-clock unchanged to within 5% at every core count. The OpenMP runtime’s barrier is already a tree, and the barrier itself dominates the scan.
2. **Gram–Schmidt is vector-bound, not reduction-bound.** MGS re-reads column $\mathbf{V}_i$ once for every $j \geq i$, so $m(m + 1)/2$ column reads at $8N$ bytes each. That quadratic is a *bandwidth* term, and it is what the roll-off in the scaling study is actually made of.

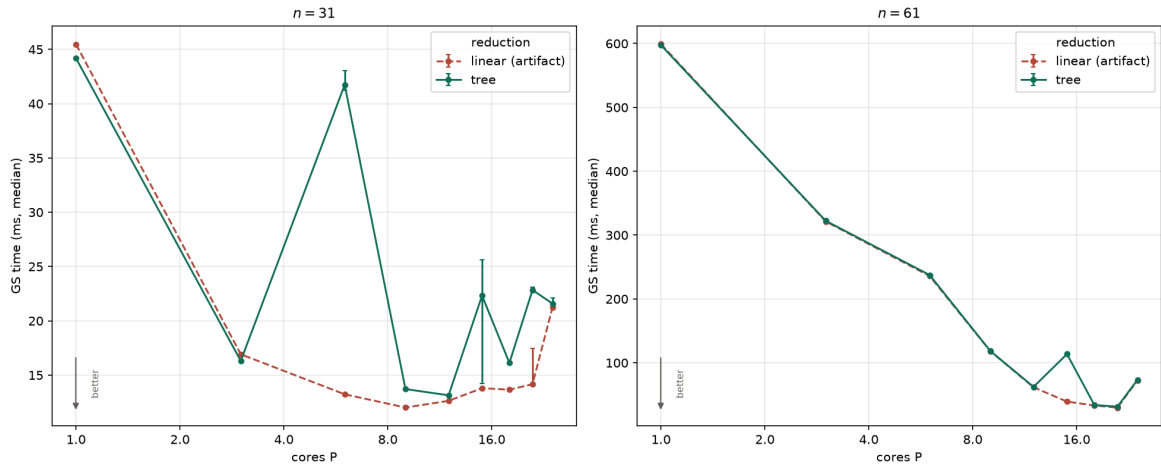


Figure 11: Gram–Schmidt wall-clock under the calibrated tree reduction vs. the $O(P)$ redundant scan it replaced ($n = 61$, arm B; one contaminated $P = 15$ launch, inflated across all kernels, is excluded). A 1.7 $\times$ faster primitive leaves the kernel unchanged within 5%.

The map predicted this before the experiment: at production resolution $n = 61$ the real operator sits at $R_h = 0.080 \ll \theta_h = 1$. The horizontal mechanism has no home on this machine, and the null result is a *confirmation* of the coordinate, not a failure of it. The thesis leads with the timed number, not the reduction count.

The upper-right corner—both mechanisms binding simultaneously—is structurally unreachable on puffin. It is a single-NUMA socket, so no tree in the reduction crosses anything more expensive than a CCX boundary; reaching θ_h at a working set that also exceeds θ_v needs a reduction cost roughly 25 $\times$ higher than any this machine can produce.

8.9 THE VERTICAL MECHANISM

The horizontal verdict came almost for free, but the vertical one requires its own experiment. The roofline of Section 6 says the spilled SpMV has $4\text{--}8\times$ of recoverable traffic, yet (6.0.1) is only a ceiling: it prices the bytes tiling would save, not the redundant halo arithmetic tiling spends to save them, so whether a real matrix-powers kernel comes out ahead is a question only a timed run can answer. A timed sweep therefore runs a cache-blocked matrix-powers kernel against the untiled baseline across grids that carry R_v from 0.035 to 86. The measurement is not close to the modeled ceiling (Table 12, Figure 12).

Table 12: The vertical sweep: measured speedup of the tiled matrix-powers kernel against the modeled traffic ratio.

Pattern	Tile level	R_v (1 core)	Measured speedup	Modeled ratio
banded	L3	0.035–2.69	0.72–1.31 $\times$	4–8 $\times$
banded	L2	1.12–86.1	0.37–0.89 $\times$	4–8 $\times$
scattered	L3	0.035–2.69	1.00 $\times$ (falls back to naive)	4 $\times$

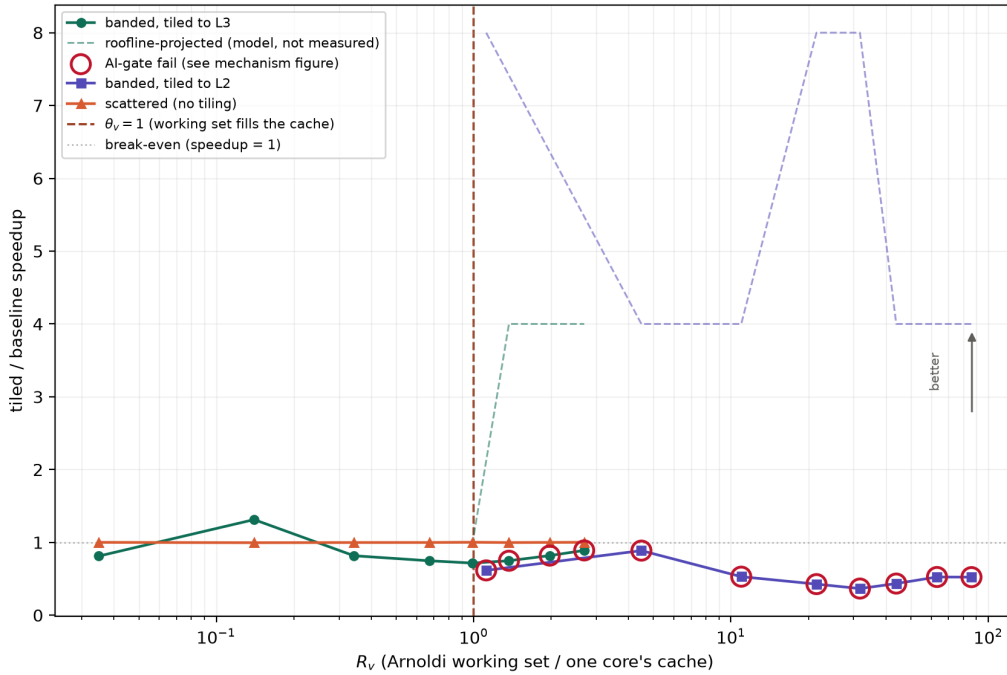


Figure 12: The vertical sweep: measured speedup of cache-blocked matrix powers across R_v . The projected curve is an explicit model claim, drawn dashed, never presented as data.

The shortfall is a **kernel ceiling, not a refuted boundary**: the tiled arm never reached a bandwidth-bound regime, so the experiment never got to test θ_v . A diagnostic kernel that skips the halo entirely (wrong basis, throughput only) separates the two tile levels cleanly (Figure 13). At L2 (small panels, fat halo), removing the halo recovers throughput toward baseline (1.17 rising to 2.13 against 2.70 at $n_1 = 400$): cache-blocking works, and the shortfall is redundant halo arithmetic traded for DRAM traffic at a loss. At L3 (large panels, thin halo), removing the halo barely moves throughput (2.06 to 2.11 against 2.76): neither arm is bandwidth-bound, and the cap is insensitive to cache residency—most consistent with gather latency on the indirect column access.

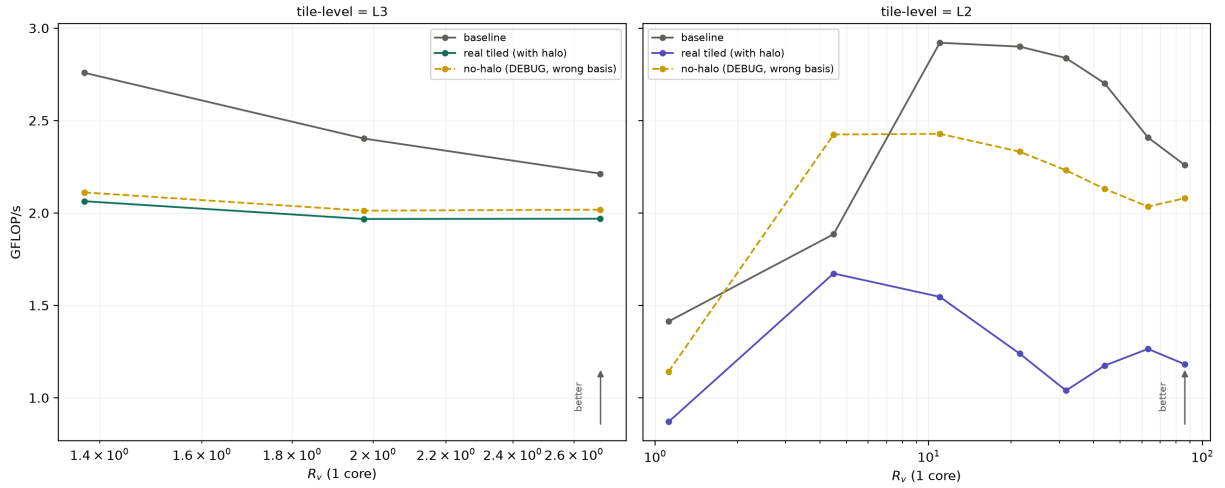


Figure 13: The no-halo diagnostic. L2 tiling recovers toward baseline without the halo (the halo trade is the shortfall); L3 tiling barely moves (the cap is insensitive to cache residency).

The scattered arm delimits where the vertical mechanism cannot pay at all. Matrix-powers tiling is structurally impossible once the columns scatter, so the sweep falls back to the naive form and measures $1.00\times$ by construction. Since the scatter knob is a similarity transform (gate C2), that boundary is a property of the sparsity pattern alone, at fixed spectrum and fixed working set.

Two honest limitations. Nothing here is a *counted* DRAM measurement, because puffin’s uncore counters need root, so modeled bytes are reconciled against the roofline instead—*weaker evidence*, labeled as such on every figure. And the projected curve on Figure 12 is a model claim, drawn dashed.

8.10 BLOCK WIDTH

Everywhere else the block width s is pinned at the certified value, which hides a question. s is pushed *up* by both mechanisms—a wider block means fewer reductions and more powers per operator stream—and pulled *down* by two independent ceilings:

- **numerical (the certificate):** $\kappa([\mathbf{v}, \mathbf{A}\mathbf{v}, \dots, \mathbf{A}^s\mathbf{v}])$ grows geometrically in s ; past $u^{-1/2}$ the monomial basis leaves the CholeskyQR certificate, so the block builds but will not orthogonalize.
- **capacity (the ghost):** the halo is $s \cdot w$ rows per side, so a wider block shrinks the panel until none fits and the tiling collapses.

Which arrives first is a joint property of operator and machine, and on puffin it depends on the tiling level (Table 13, Figure 14).

Table 13: The two roofs on the block width. The L3 ghost roof is quoted as $\geq$ because a panel still fit at every swept s up to the sweep ceiling of 24; the certificate roof is measured directly ($\kappa(\mathbf{B}_s)$ crosses $u^{-1/2}$ between $s = 8$ and $s = 9$ at $n_1 = 420$).

Tile level	n_1	Certificate roof s	Ghost roof s	Binding roof
L3	420	8	≥ 24	numerical
L3	480, 560	6	≥ 24	numerical
L2	420	8	3	capacity
L2	480	6	3	capacity
L2	560	6	2	capacity

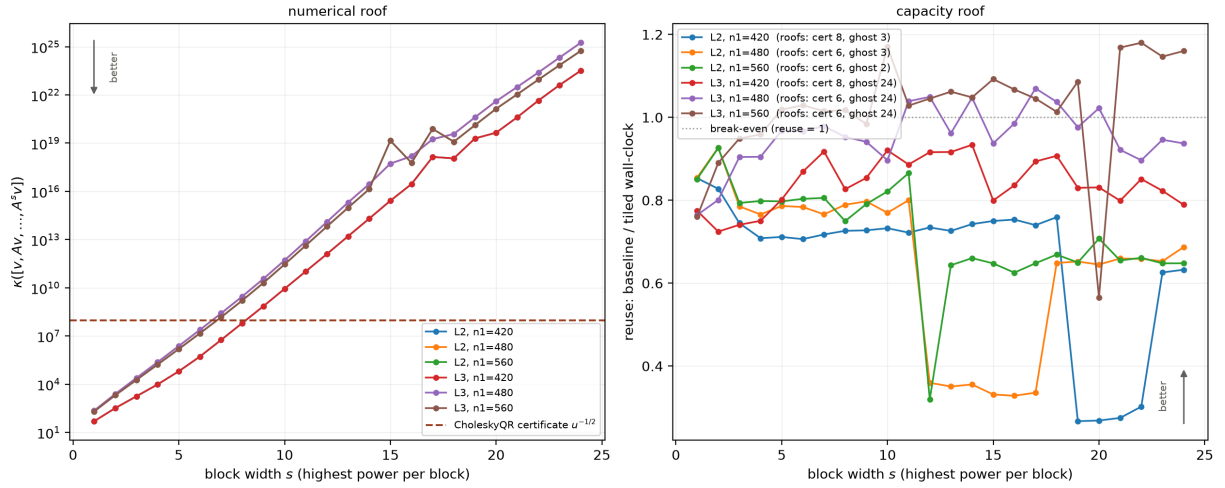


Figure 14: The two roofs on the block width s : the numerical certificate and the ghost-zone capacity, per tiling level.

The distinction is actionable, because the two roofs respond to different investments. The certificate roof is set by how fast $\kappa(\mathbf{B}_s)$ grows per power, and that growth rate is a property of the polynomial basis: replacing the monomial recurrence with a Newton basis (shifts at Leja-ordered Ritz values) or a Chebyshev basis keeps the growth near flat, a standard remedy in the s -step literature [2, 15, 6]. So where the certificate binds (L3 tiling here), a better basis converts the ghost roof’s slack into real reductions. Where the ghost binds (L2 tiling), the roof is set by geometry—halo rows per panel—and only a thinner operator bandwidth or a coarser tiling level moves it; a better basis buys nothing. Pinning s at the certified value assumes the first case without measuring which one holds.

8.11 PLACING THE REAL OPERATOR ON THE MAP

The measured real-operator points anchor the map; a predicted continuation carries the operator to production grids. The continuation uses the *same* a-priori placement the instrument uses, and it is validated against the measured coordinates before it is trusted to extrapolate—the replica reproduces every measured (R_v, R_h) to better than 0.001% (Table 14, Figure 15).

Table 14: The real operator’s trajectory under grid refinement. Bisecting on n puts the θ_v crossing at $n \approx 73$.

n	N	m	R_v	R_h	Source
10	1,000	6	0.0019	13.53	measured
20	8,000	8	0.0175	1.87	measured
25	15,625	8	0.036	0.904	predicted
61	226,981	12	0.582	0.080	predicted
74	405,224	13	1.063	0.047	predicted (first above θ_v)
120	1,728,000	15	4.74	0.0025	predicted

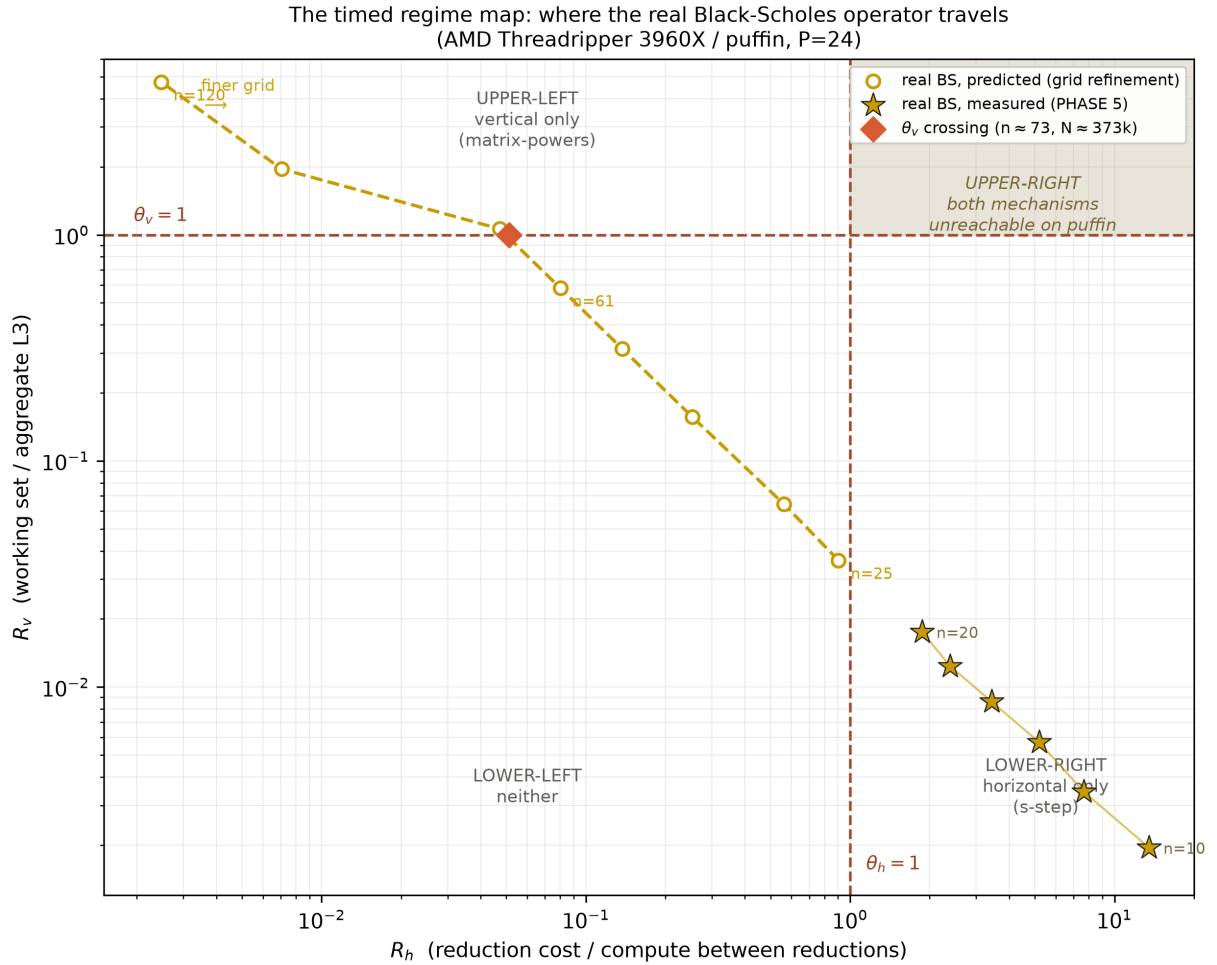


Figure 15: The regime map. The real Black-Scholes operator’s refinement trajectory travels lower-right $\rightarrow$ lower-left $\rightarrow$ upper-left, crossing θ_v just under $n = 74$ and never entering the upper-right corner.

In summary: the early $R_h > 1$ readings occur only on grids too small for any desk to price at ($n \leq 20$, the dense-eigensolvable range), so they mark no regime where s -step pays. At production resolution the operator is squarely in **vertical-only** territory—the mechanism the application needs is matrix-powers tiling, while the mechanism the certificate work supports is s -step, and they are not the same corner. The upper-right corner is a portability statement on this hardware rather than a measurement: reaching it requires a reduction cost roughly $25\times$ puffin’s, i.e. crossing a device or node boundary rather than a CCX boundary.

That $25\times$ is not exotic—any machine with multiple NUMA nodes or a multi-socket, multi-node topology supplies it, and on such CPU systems the upper-right corner is accessible and already well charted. Reducing the cost of global synchronization on distributed and complex topologies is a mature literature: s -step methods were introduced precisely for that setting [7], the communication-avoiding Krylov program quantified their message and latency savings for MPI-class machines [8, 15, 6, 3], and pipelined variants hide the reduction latency the topology imposes [11]. All of that work, however, lives on *homogeneous* compute: every core sees the same kind of cache hierarchy the map was built on. What remains uncharted is a memory system that differs in kind, and the cluster this study targets next—“synge”, two nodes of two Tesla V100s each, GPUs joined within a node by PCIe and nodes joined by InfiniBand—offers exactly that combination: each interconnect rung adds enough global-reduction latency to open the upper-right corner, on a memory system where the map’s coordinates have never been recomputed. That recomputation is the subject of the next section.

9 RECOMPUTING THE MAP FOR A GPU

9.1 THE PORTABILITY CLAIM UNDER TEST

The map’s central claim is that R_v and R_h are *dimensionless*—ratios, not times—so a reader can place their own operator and machine on the map. Within this thesis the claim is tested on one memory system only: puffin is one socket, one NUMA node, one cache hierarchy. A GPU memory system differs in kind, not in constants, so recomputing the coordinates for one is not a port: it is a genuine test of the dimensionless framing, and both outcomes are informative. Either coordinates recomputed from GPU constants predict a measured GPU crossover at $\theta_v = \theta_h = 1$, or they do not—in which case the map is scoped to CPU-like memory systems and the failure names the missing term. This section derives the structural content of that recomputation from the CPU study’s own results.

Two GPU classes make a near-controlled pair, and the pair is a sharper instrument than either card alone (Table 15): a consumer card and a datacenter card whose memory systems are nearly identical—same 6 MB L2, same bandwidth class—while the double-precision (FP64) rate differs by $\sim 11\times$. The pair holds the map’s vertical axis roughly fixed and varies only the compute roof.

Table 15: The two-GPU controlled pair, calibrated on idle devices. Memory bandwidth is achieved (STREAM); the FP64 ridge is peak FP64 over achieved bandwidth. The pair holds L2 and bandwidth nearly fixed and moves the compute roof by $\sim 11\times$.

	L2	Memory BW	FP64 peak	FP64 ridge
RTX 3090 (consumer, puffin)	6 MB	821 GB/s	0.570 TFLOP/s	0.69 FLOP/B
Tesla V100 (datacenter, synge)	6 MB	818 GB/s	6.375 TFLOP/s	7.79 FLOP/B

That lever exposes a precondition the map has so far carried implicitly. Both axes are *communication* axes, presupposing the kernel is on the memory side of the roofline; a hard-throttled FP64 rate does not move a point around the map, it throws the point *off* the map into compute-bound territory neither coordinate charts. Placement must therefore be gated by an explicit roofline check— $AI \cdot BW_{\text{achieved}} < \text{FLOP}/s_{\text{peak}}$ in the relevant precision—evaluated before the map is read. The gate turns the two-card pair into a designed experiment: from the cards’ constants alone it should pass on the datacenter card (FP64 memory-bound) and fail on the consumer card (FP64 compute-bound), so the consumer card is a negative arm rather than a hazard. A map that can state where it stops applying is a stronger form of the dimensionless claim than any single-machine crossover.

9.2 WHAT TRANSFERS, AND WHAT BREAKS

Everything machine-independent transfers unchanged: the synthetic operator, the tensor identity (8.6.1), the numerics gates C1–C6, the reduction ledger, the CholeskyQR2 certificate, and the thresholds $\theta_v = \theta_h = 1$ themselves—a ratio crosses 1 where supply meets demand, which is what makes the thresholds portable. Only the coordinates are recomputed. Three things break structurally, not by constant swap:

- **The denominator of R_v .** On the CPU each engaged CCX brings its own L3 slice, so aggregate last-level cache scales with the team. A GPU’s L2 is a single fixed block shared by every streaming multiprocessor: engaging more of the device adds zero cache, so within one device R_v stops being a function of the team size at all. This decouples the two axes. On the CPU both coordinates shared the parallelism knob ($R_v \sim 1/P$, $R_h \sim P \log P$), which is why $R_v \cdot R_h$ collapsed to a P -independent constant (~ 0.04 on puffin)—the invariant that made the upper-right corner unreachable on one node. On a GPU that cancellation has no structural reason to survive, so whether the product remains an invariant must be re-derived from the chosen definition of team size, not assumed.
- **The numerator of R_h .** The CPU model has two reduction tiers and one saturation constant (8.1.1). A GPU reduction crosses at least five qualitatively different tiers (Table 16), and the grid tier carries kernel-launch latency, which has no CPU analogue—a fixed cost per reduction independent of tree depth. Two parameters cannot express five tiers: the machine model needs a tier vector, and the reduction-cost predictor needs to walk it. The ladder is the instrument, not an obstacle: three reachable rungs with distinct costs (on-device, intra-node across PCIe, inter-node across the fabric) place the *same* problem at three different R_h values, so holding the problem and device fixed while changing only the reduction hardware traverses the horizontal axis directly, converting θ_h from a shaded prediction into a measured crossing.
- **The reuse model.** The CPU scatter knob costs a factor of a few in cache-line reuse; on a GPU the same permutation defeats warp coalescing and can serialize a 32-thread gather into 32 transactions. The scatter axis becomes *more* discriminating, but the reuse model must be recalibrated from scratch.

Table 16: The reduction-tier ladder for a multi-node GPU cluster with a PCIe intra-node interconnect. Every tier cost is a machine constant to be calibrated, never assumed.

Tier	Mechanism	Scope	Cost class
Warp	register shuffle	on-die	near-free, no memory traffic
Block	shared memory + barrier	on-die	cheap, on-chip
Grid	atomics or a second launch	on-die	expensive; carries launch latency
Device	PCIe + cross-socket link	intra-node	expensive; a genuine cross-socket hop
Node	network fabric	inter-node	dominant

9.3 THE DECOUPLED INVARIANT

On the CPU the product $R_v \cdot R_h$ is independent of the core count P by *cancellation*: $R_v \sim 1/P$ and $R_h \sim P \log P$ pull in opposite directions, so P slides a point along the map’s anti-diagonal but

can never lift it off—which is exactly what made the upper-right corner unreachable on one socket. Recomputing the coordinates for a GPU replaces that cancellation with something stronger. Take P to be the number of streaming multiprocessors (SM) engaged, the unit that carries its own FP64 pipelines and so the GPU analogue of a core. Neither coordinate then depends on it: R_v 's denominator is the single fixed L2, so engaging more SMs adds no cache, and R_h 's denominator is the device-wide bandwidth roof, flat once saturated. The axes are not cancelling but *decoupled*—the grid size N moves R_v alone, the reduction tier moves R_h alone, and P moves neither. In the memory-bound regime the roofline gate certifies, the modeled cycle time collapses to bytes over bandwidth and the product takes the closed form

$$R_v \cdot R_h = g(m) R(m) \Lambda, \quad \Lambda = \frac{\tau \text{ BW}}{C}, \quad (9.3.1)$$

in which $g(m)$ (a byte ratio) and $R(m)$ (the MGS reduction count) are the machine-independent method terms, and Λ —the only place the machine enters—is dimensionless and reads as how many cache-fulls of data the memory system delivers during one reduction. Two consequences follow at once. First, the upper-right window (both $R_v \geq 1$ and $R_h \geq 1$ at a footprint under the 16 GB device memory) is non-empty exactly when this product exceeds 1, and its width in N is the product: the two questions of whether a corner exists and how wide it is collapse to one. Second, the 16 GB ceiling never binds—it is out by two orders of magnitude at every reachable tier—so the constraint that closes the window from above is R_h , the reduction cost, not capacity. Setting the product to 1 and solving for the reduction latency gives the recomputation's single sharpest prediction: a reduction costing more than $\tau^* \approx 0.9 \mu\text{s}$ carries the production operator into the corner.

That threshold is measured, not assumed. Calibrating the V100's full reduction ladder—each rung timed as a bare all-reduce over an empty team, no payload—places the same operator at five different R_h values by changing only the reduction hardware (Table 17). The corner opens between the block and grid rungs, at a measured $\tau^* = 0.92 \mu\text{s}$, and that boundary falls *inside a single device*: the on-device grid reduction (a second kernel launch) already clears it, so the upper-right corner is reachable without leaving the GPU. Two features of the ladder were not anticipated. The fabric step is shallow—node-to-node costs only $1.9\times$ device-to-device, against the CPU study's $5.4\times$ for a single cache-domain crossing—because synge's intra-node link is PCIe plus a cross-socket hop with no NVLink, slow enough that leaving the node barely doubles it; the horizontal axis is effectively {on-device, off-device}, with the fabric a modest extension rather than a new decade. And the widest single step is grid over block at $7\times$, most of it kernel-launch latency—a fixed software cost with no CPU analogue, so the ladder's biggest jump is not an interconnect at all.

Table 17: The V100 reduction ladder, calibrated. Each rung is a bare all-reduce and τ is cumulative. The corner opens ($R_v \cdot R_h > 1$) between the block and grid rungs—inside a single device.

Rung	Scope	τ (cumulative)	Corner
warp (register shuffle)	on-die	$0.42 \mu\text{s}$	closed
block (shared + barrier)	on-die	$0.79 \mu\text{s}$	closed
grid (second kernel launch)	on-die	$5.54 \mu\text{s}$	open
device (PCIe + cross-socket)	intra-node	$11.46 \mu\text{s}$	open
node (inter-node fabric)	inter-node	$22.09 \mu\text{s}$	open

9.4 REACHING THE CORNER, AND THE GATE THAT DISCRIMINATES

At production resolution ($n_1 = 61$, $m = 12$, $s = 8$) the recomputed coordinates place the operator at $R_v = 7.2$, $R_h = 1.7$ on *both* cards—for the 3090 at its on-device grid rung, for the V100 at the device-to-device rung—squarely in the upper-right corner the single-NUMA CPU was structurally barred from. The dimensionless framing transfers in the strongest sense available: the thresholds $\theta_v = \theta_h = 1$

did not move, only the coordinates were recomputed from constants measured on a memory system that differs in kind, and the operator landed where the recomputation said it would.

Reaching the corner is only half the test, because both axes presuppose the kernel is on the memory side of the roofline. The pair was built to probe exactly that precondition: with near-identical L2 and bandwidth and an $11\times$ gap in FP64 rate, the gate should—from the cards’ constants alone—pass on the datacenter card and fail on the consumer one, and it fires precisely where the a-priori gate analysis said it would (Table 18). The baseline method is on-map on both cards: SpMV (arithmetic intensity 0.135 FLOP/B) and MGS (0.375) sit so far below even the 3090’s throttled ridge of 0.69 that a 1:66 FP64 penalty is irrelevant to them—so the negative arm as one might first state it, “CA is pointless on the 3090 because its FP64 is throttled,” would not have fired on the baseline kernels at all. What separates the cards is the tall-skinny Gram matrix $\mathbf{B}^T\mathbf{B}$ that the s -step arm introduces and MGS does not have: its intensity grows with the block width ($\sim s/4$) and crosses the 3090’s ridge at $s \approx 2.4$, well below the certified $s_{\max}$. At the production width $s = 8$ that same kernel is memory-bound on the V100 (a $3.9\times$ margin) and compute-bound on the 3090. Both cards exhibit the corner; the map predicts that only one of them can pay for it. A map that states where it stops applying, and is right, is a stronger result than any single-machine crossover.

Table 18: The roofline gate at production resolution, per kernel. The baseline kernels are on-map on both cards; the s -step Gram block is memory-bound only on the datacenter card.

Kernel (AI, FLOP/B)	V100 (ridge 7.79)	3090 (ridge 0.69)
SpMV (0.135)	on map	on map
MGS (0.375)	on map	on map
s -step Gram (~ 1.1)	memory-bound ($3.9\times$)	compute-bound

9.5 ROOFLINE-BOUND KERNELS AND THE s -STEP CROSSOVER

Reading the gate on the real kernel required first making that kernel efficient, and the vendor library could not. The s -step block orthogonalization is CholeskyQR: a Gram matrix ($\mathbf{B}^T\mathbf{B}$, a `syrk`), a Cholesky factorization (`potrf`), and a triangular solve ($\mathbf{B} \leftarrow \mathbf{B}\mathbf{R}^{-1}$, a `trsm`). For an $n \times s$ block with $s \leq 9$ both level-3 BLAS operations are *tall and skinny*—a huge $k = n$ against a tiny $s \times s$ —and cuBLAS, built for square matrices, serves them badly: its `dsyrk` ran at ~ 0.7 GFLOP/s (under 0.1% of roof, latency-bound on a handful of output threads) and its `dtrsm` was compute-bound at $\sim 2\%$ useful FP64. The map’s mechanism was invisible behind library inefficiency, so two kernels were written: a split-K Gram that parallelizes the $k = n$ dimension and reduces the partials [10], and a per-row triangular solve exploiting that the m row-solves are independent. Both are validated bit-close to cuBLAS and reach the roofline (Table 19)—the split-K Gram roughly $1900\times$ faster than cuBLAS, the per-row solve $27\times$.

Table 19: The two hand-written tall-skinny kernels at the production block width. Both reach the roofline; the two-card split appears on the Gram exactly as the gate predicts.

Kernel	vs. cuBLAS	V100	3090
split-K Gram	$\sim 1900\times$	85% DRAM roof (memory-bound)	71% FP64 peak (compute-bound)
per-row trsm	$27\times$	95% DRAM roof (memory-bound)	94% roof (at the knee)

With roofline-bound kernels the two-card contrast becomes a hardware measurement rather than a prediction. Profiled with Nsight Compute at $s = 8$, $n = 2 \times 10^6$, the *same* Gram kernel runs at 86% of DRAM throughput and 16% of compute on the V100 (memory-bound, FP64 idle) and at 38% of

DRAM against 94% of compute on the 3090 (compute-bound). The triangular solve, whose intensity (~ 0.56) sits just below the 3090's ridge, is memory-bound on the V100 and at the compute/memory knee on the 3090, so the Gram is the sole unambiguous carrier of the negative arm—exactly the kernel the roofline gate named. On consumer silicon the CA treatment's own orthogonalization kernel is compute-bound while the baseline it replaces is memory-bound on both cards.

The reduction saving itself was then timed. Against MGS at matched $m = 12$, the s -step block—orthogonality $\|\mathbf{I} - \mathbf{Q}^T \mathbf{Q}\| \sim 10^{-15}$ throughout—trades MGS's $1 + m(m + 3)/2 = 91$ reductions for $1 + 2\lceil m/s \rceil = 5$. On a single device, with the roofline-bound kernels, s -step beats MGS at every grid on both cards. But an on-device reduction is cheap, and the horizontal threshold θ_h by definition needs an expensive one, so the crossing was measured across two GPUs: the n basis rows split across the pair, and every MGS dot and every s -step Gram becomes a real NCCL all-reduce over the device-to-device link. That all-reduce measured $\sim 11.5 \mu\text{s}$, matching the calibrated device rung ($11.46 \mu\text{s}$) and cross-checked independently by a two-GPU MGS decomposition and an Nsight Systems trace. At that rung the s -step block's five reductions beat MGS's ninety-one across the whole production range of n : the horizontal saving that moved nothing on the single CPU socket pays once the reduction crosses a real device boundary—the upper-right corner, entered and collected. The instruments here are the orthogonalizer and its reductions, not an end-to-end priced solve; what the two-card pair establishes is that coordinates recomputed from GPU constants place, and correctly gate, the same operator the CPU study could only argue about.

10 CONCLUSION

This thesis set out to decide when communication-avoiding reformulations of a Krylov exponential integrator pay for a multi-asset option-pricing problem, and to make the answer portable. The instrument for that answer is a dimensionless regime map whose coordinates are predictors—computed from the problem, the spectrum, and machine constants before any run—and whose boundaries are tested by measurement.

On the numerical side, the certified s -step block width is predicted from the spectrum alone and verified integer-exact on 20 of 21 runs; a confound control shows safety is a property of the spectrum, not of the Krylov dimension; an empirically fitted log-robustness relation bounds the damage non-normality does to the prediction (slope 0.069 in decades per decade of eigenvector conditioning for the constant-coefficient family, $4\times$ steeper for variable coefficients, the real operator under one s -step everywhere); and a closed-form tensor identity gives the geometric growth of $\kappa(X)$ with asset dimension, along with the warning that the dense eigensolver value is meaningless once the spectrum degenerates.

On the measurement side, the study delivers three placed results. The horizontal mechanism is a null result with a mechanism: an $11\text{--}16\times$ cut in reduction count moves wall-clock by nothing, because the reduction primitive was never the bottleneck and Gram-Schmidt is bandwidth-bound—exactly as the map's $R_h = 0.080$ predicted. The vertical mechanism is a shortfall with a mechanism: cache-blocked matrix powers loses to its own halo arithmetic at L2 tile sizes and to a residency-insensitive cap at L3, so the predicted crossover θ_v remains untested by this kernel rather than refuted. And the real operator's trajectory under refinement—lower-right, lower-left, upper-left, crossing θ_v near $n = 74$ —says the mechanism the application needs at production resolution is matrix-powers tiling, not s -step, and that the upper-right corner is structurally unreachable on a single-NUMA socket.

The map's central claim—that its coordinates transfer—was also its least-tested one, and Section 9 put it on a GPU memory system that differs in kind. The recomputation is structural, not a constant swap: a fixed shared L2 replaces a scaling aggregate cache in R_v 's denominator, which decouples the two axes that cancelled on the CPU; a five-tier reduction ladder replaces two tiers in R_h 's numerator;

and an explicit roofline gate must precede any placement. Carried out on a controlled pair of GPUs—a datacenter V100 and a consumer 3090, near-identical in memory and $11\times$ apart in FP64—the recomputed coordinates place the production operator in the upper-right corner the single-socket CPU was barred from, and the gate correctly predicts that the s -step arm’s own orthogonalization kernel, hand-written to reach the roofline, is memory-bound on the datacenter card but compute-bound on the consumer one. The reduction saving that moved nothing on the CPU pays once the reduction crosses a real device boundary. The thresholds never moved; only the coordinates were recomputed, and the operator landed where they said—the dimensionless framing vindicated on hardware it had never seen.

BIBLIOGRAPHY

- [1] Awad H. Al-Mohy and Nicholas J. Higham. Computing the action of the matrix exponential, with an application to exponential integrators. *SIAM Journal on Scientific Computing*, 33(2):488–511, 2011.
- [2] Zhaojun Bai, Dan Hu, and Lothar Reichel. A Newton basis GMRES implementation. *IMA Journal of Numerical Analysis*, 14(4):563–581, 1994.
- [3] Grey Ballard, James Demmel, Olga Holtz, and Oded Schwartz. Minimizing communication in numerical linear algebra. *SIAM Journal on Matrix Analysis and Applications*, 32(3):866–901, 2011.
- [4] Mario Berljafa and Stefan Güttel. Parallelization of the rational Arnoldi algorithm. *SIAM Journal on Scientific Computing*, 39(5):S197–S221, 2017.
- [5] Fischer Black and Myron Scholes. The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3):637–654, 1973.
- [6] Erin Carson. *Communication-Avoiding Krylov Subspace Methods in Theory and Practice*. PhD thesis, University of California, Berkeley, 2015.
- [7] Anthony T. Chronopoulos and C. William Gear. s -step iterative methods for symmetric linear systems. *Journal of Computational and Applied Mathematics*, 25(2):153–168, 1989.
- [8] James Demmel, Mark Hoemmen, Marghoob Mohiyuddin, and Katherine Yelick. Avoiding communication in sparse matrix computations. Technical report, University of California, Berkeley, 2008. IEEE International Symposium on Parallel and Distributed Processing.
- [9] Jim Douglas and Henry H. Rachford. On the numerical solution of heat conduction problems in two and three space variables. *Transactions of the American Mathematical Society*, 82:421–439, 1956.
- [10] Dominik Ernst, Georg Hager, Jonas Thies, and Gerhard Wellein. Performance engineering for real and complex tall & skinny matrix multiplication kernels on GPUs. *International Journal of High Performance Computing Applications*, 35(1):5–19, 2021.
- [11] Pieter Ghysels and Wim Vanroose. Hiding global synchronization latency in the preconditioned conjugate gradient algorithm. *Parallel Computing*, 40(7):224–238, 2014.
- [12] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, 4th edition, 2013.
- [13] Nicholas J. Higham. The scaling and squaring method for the matrix exponential revisited. *SIAM Journal on Matrix Analysis and Applications*, 26(4):1179–1193, 2005.

- [14] Marlis Hochbruck and Christian Lubich. On Krylov subspace approximations to the matrix exponential operator. *SIAM Journal on Numerical Analysis*, 34(5):1911–1925, 1997.
- [15] Mark Hoemmen. *Communication-Avoiding Krylov Subspace Methods*. PhD thesis, University of California, Berkeley, 2010.
- [16] Willem Hundsdorfer and Jan G. Verwer. *Numerical Solution of Time-Dependent Advection-Diffusion-Reaction Equations*, volume 33 of *Springer Series in Computational Mathematics*. Springer, 2003.
- [17] Karel J. in 't Hout and Sven Foulon. ADI finite difference schemes for option pricing in the Heston model with correlation. *International Journal of Numerical Analysis and Modeling*, 7(2):303–320, 2010.
- [18] Marghoob Mohiyuddin, Mark Hoemmen, James Demmel, and Katherine Yelick. Minimizing communication in sparse matrix solvers. In *Proceedings of the Conference on High Performance Computing Networking, Storage and Analysis (SC '09)*, 2009.
- [19] Jitse Niesen and Will M. Wright. Algorithm 919: A Krylov subspace algorithm for evaluating the φ -functions appearing in exponential integrators. *ACM Transactions on Mathematical Software*, 38(3):22:1–22:19, 2012.
- [20] Yousef Saad. Analysis of some Krylov subspace approximations to the matrix exponential operator. *SIAM Journal on Numerical Analysis*, 29(1):209–228, 1992.
- [21] Lloyd N. Trefethen and David Bau, III. *Numerical linear algebra*. SIAM, 6 1997.
- [22] Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.
- [23] Yusaku Yamamoto, Yuji Nakatsukasa, Yuka Yanagisawa, and Takeshi Fukaya. Roundoff error analysis of the CholeskyQR2 algorithm. *Electronic Transactions on Numerical Analysis*, 44:306–326, 2015.